RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



AD23B31 Image Processing and Computer Vision

Laboratory Record Note Book

Name : POOJA B

Year / Branch / Section : II / CSE / F

University Register No. : 2116250701518

College Roll No. : 250701518

Semester : III

Academic Year : 2025-2029

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AD23B31 Image Processing and Computer Vision

NAME	POOJA B
ROLL NO	250701518
YEAR	II
SEC	F

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: POOJA B ……

Academic Year: 2025-2029 **Semester:** III **Branch:** CSE

Register No. 2116250701518

Certified that this is the bonafide record of work done by the above student in the FUNDAMENTAL OF DATA SCIENCE CS23334 ***Laboratory during the academic year 2025 – 2026.***

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

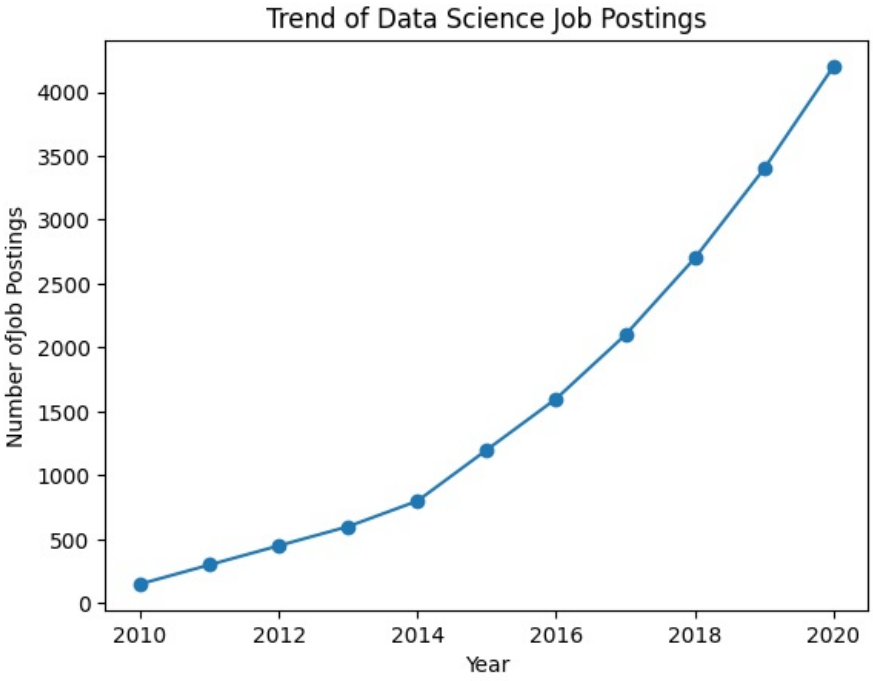
Sl No	Experiment	Date	Page No
1	Data visualization of job trends and data encryption / decryption using Fernet		5
2	Data cleaning, aggregation and visualization of sales data		7
3	Line plot comparison of cricket scores (India vs Sri Lanka)		10
4	Word tokenization and word frequency using NLTK		11
5	Exploratory data analysis of the diabetes dataset (histogram and pair plot)		12
6	Time series analysis: decomposition, moving average and Z-score		15
7	Central Limit Theorem: sampling distribution of sample means		17
8	Hypothesis testing: Z-test for a sample mean		18
9	Logistic regression with train-test split		19
10	K-means clustering on customer data		21
11	Understand pandas library use case in Data Science		24
12	Detect outliers in a given dataset		29
13	Handle missing values and inappropriate data in a given dataset		33
14	Understand feature scaling		43
15	Understand data preprocessing in Data Science		47
16	Understand EDA: quantitative and qualitative analysis		51
17	Understand Linear Regression for a given dataset		60
18	Understand KNN algorithm for a given dataset		62
19	Understand Logistic Regression for a given dataset		64
20	Understand K-means clustering algorithm for a given dataset		74

EXPERIMENT NO: 1

Program:

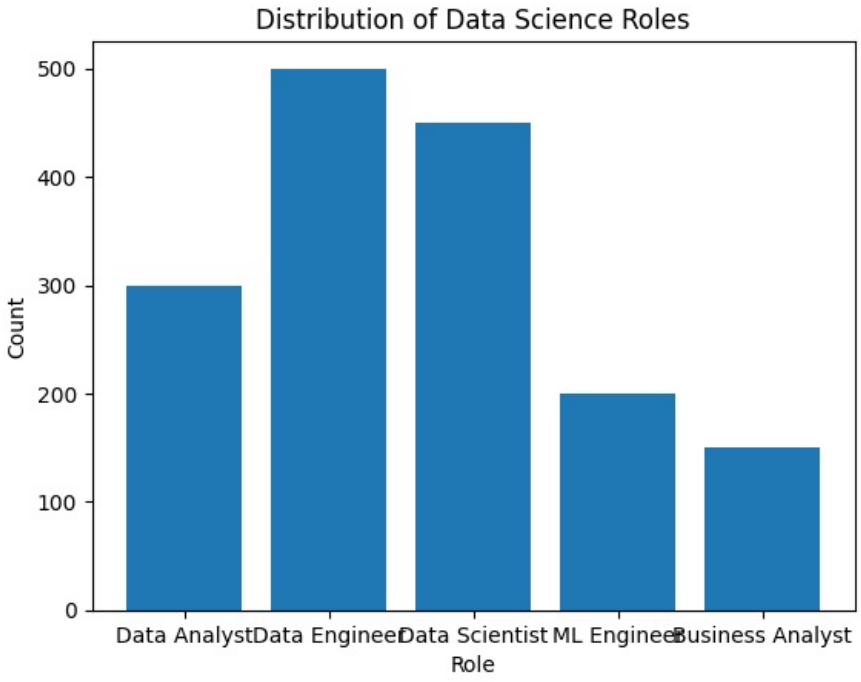
```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()
```



Output:

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
'Business Analyst']
counts = [300, 500, 450, 200, 150]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```



Output:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
```

```
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

Output:

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqyhAK3esIu7CL235sf9psDCMpGfw5En5EbEHBWkeezBbkoe8P9SDvtweqS6V6efA9QUrURy7-
Yuju0uMnkn9S7cAztoxeimsrtlxrTzA_6PD_VNdXdHt_9KT-44rVEqmTIRw9'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

EXPERIMENT NO: 2

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('sales_data.csv')

print(df.head())

print(df.isnull().sum())

df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

print(df.describe())

product_summary = df.groupby('Product').agg({
'Sales': 'sum',
'Quantity': 'sum'
}).reset_index()
print(product_summary)

11

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()

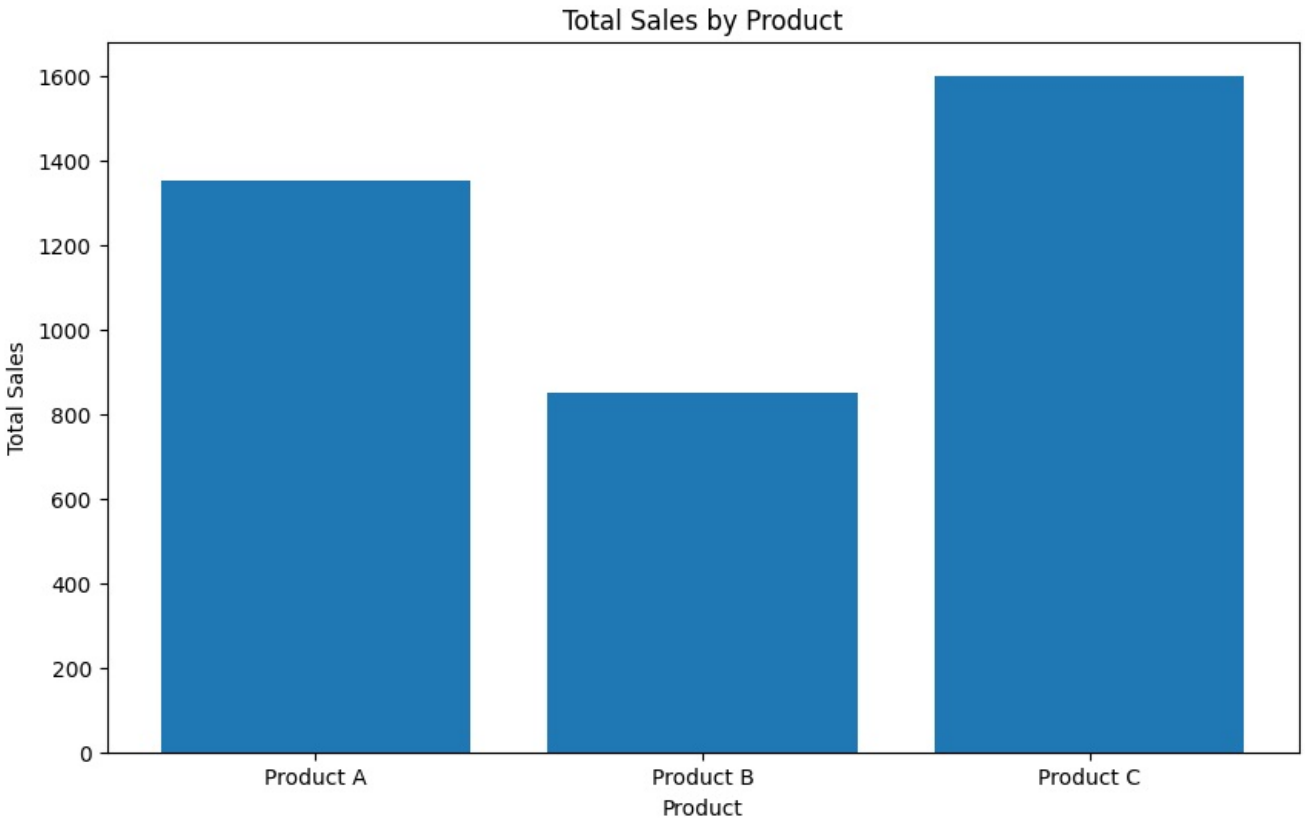
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc=np.sum, fill_value=0)
print(pivot_table)

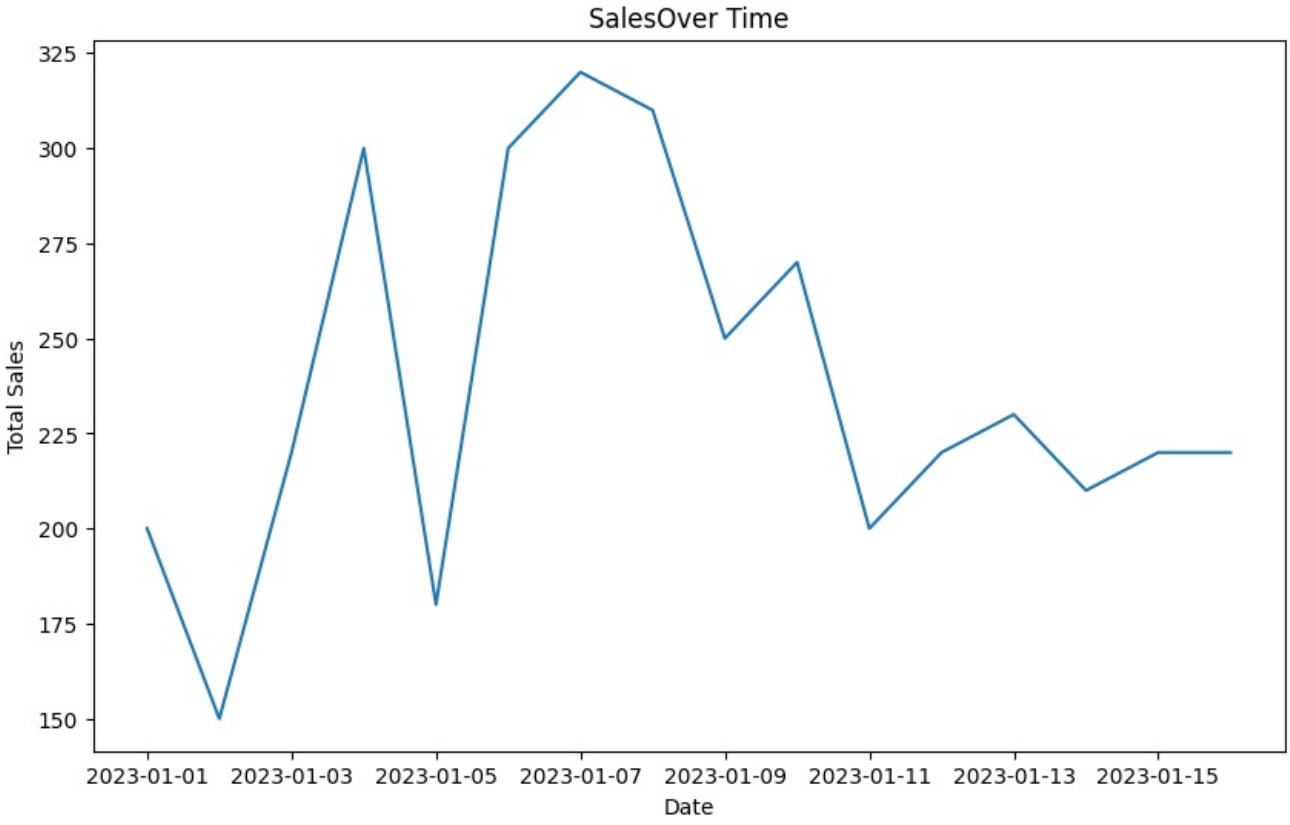
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
```

Output:

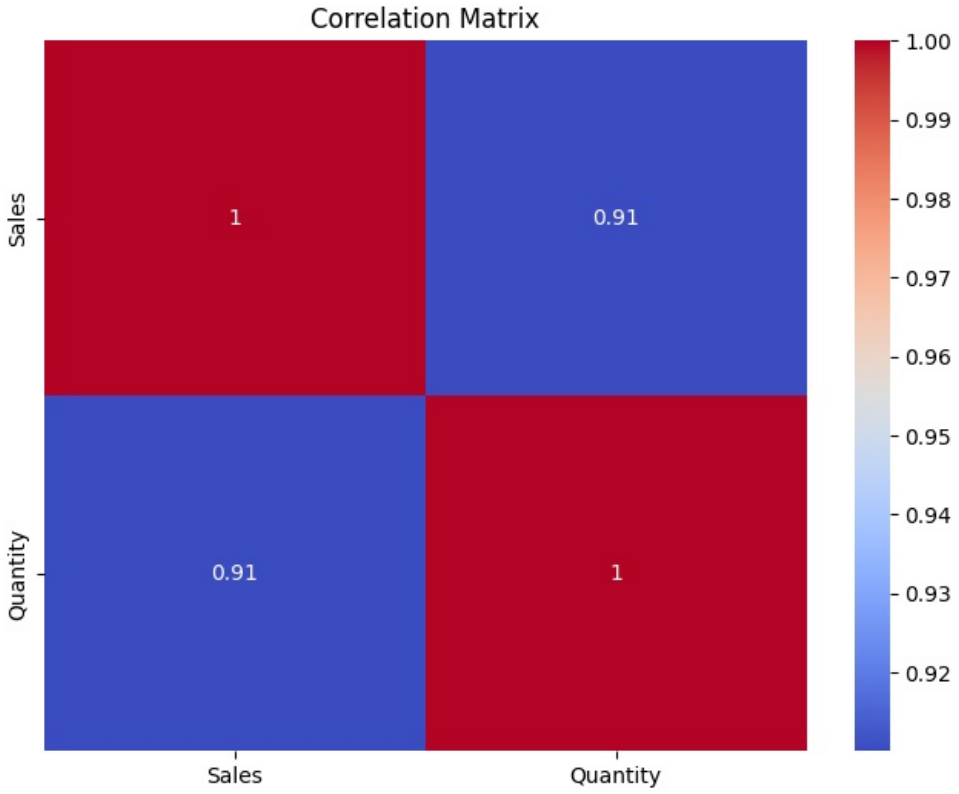
```
         Date    Product  Sales  Quantity Region
0  2023-01-01  Product A    200         4  North
1  2023-01-02  Product B    150         3  South
2  2023-01-03  Product A    220         5  North
3  2023-01-04  Product C    300         6   East
4  2023-01-05  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     49.598387   1.454877
min    150.000000   3.000000
25%    207.500000   5.000000
50%    220.000000   5.000000
75%    277.500000   5.250000
max    320.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```






```
Product  Product A  Product B  Product C
Region
East           310          0        740
North          720          0        410
South          320        420        220
West             0        430        230
              Sales  Quantity
Sales      1.000000  0.910021
Quantity   0.910021  1.000000
```

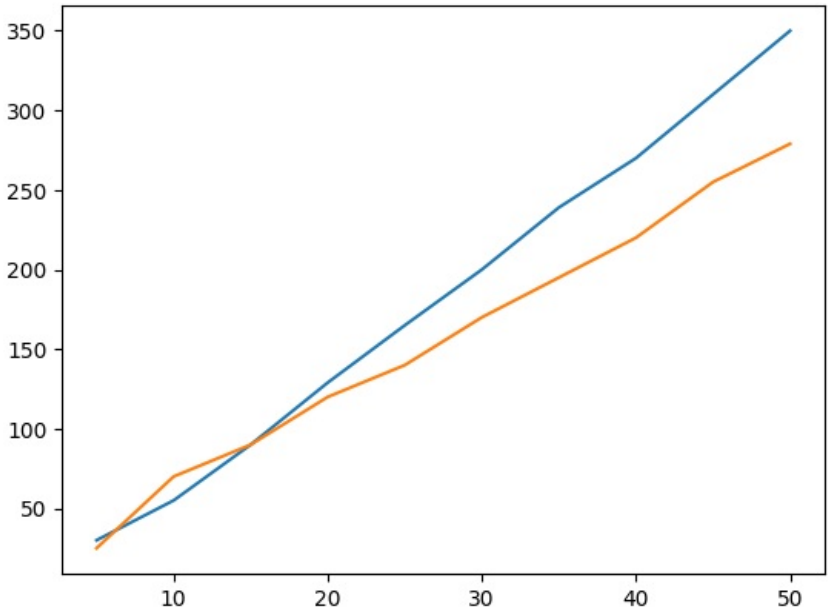
```
Text(0.5, 1.0, 'Correlation Matrix')
```


Correlation Matrix
Sales
Quantity
1
0.91
0.91
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.92

EXPERIMENT NO: 3

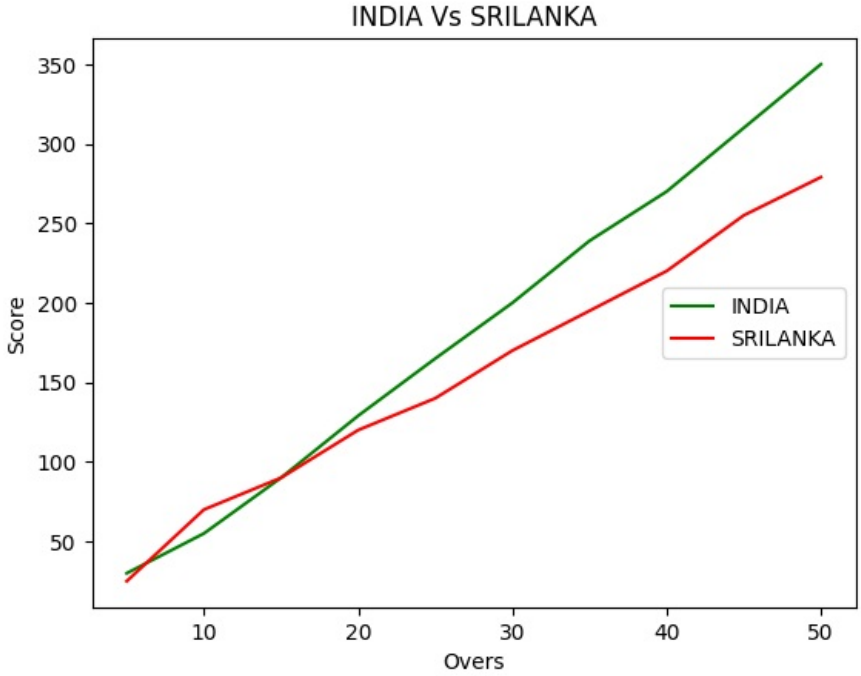
Program:

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```



Output:

```
<matplotlib.legend.Legend at 0x7c5de17cfc50>
```



EXPERIMENT NO: 4

Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')

sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)

wlist = []
for i in range(50):
    wlist.append(token[i])

wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I',
2), ('Emma', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1), (',', 5), ('and', 3), ('rich', 1), (',', 5),
('with', 2), ('a', 1), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposition', 1), (',', 5), ('seemed', 1),
('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('existence', 1), (';', 1),
('and', 3), ('had', 1), ('lived', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world', 1), ('with',
2)]
```

EXPERIMENT NO: 5

Program:

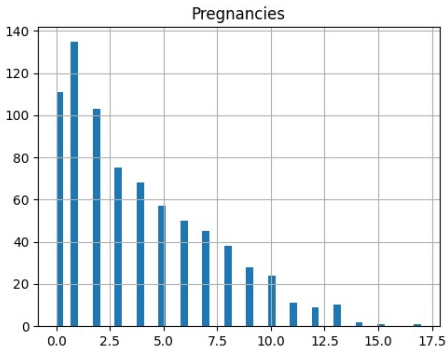
```
import pandas as pd
db=pd.read_csv('diabetes.csv')
print(db.head())
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50, figsize=(20,15))
plt.show()
sns.pairplot(db)
plt.show()
```

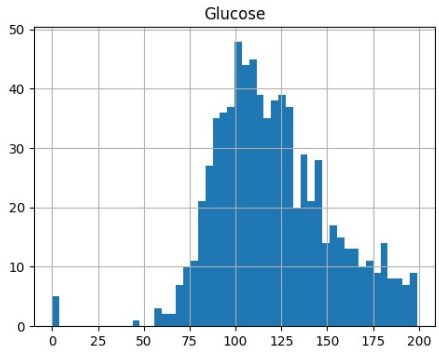
Output:

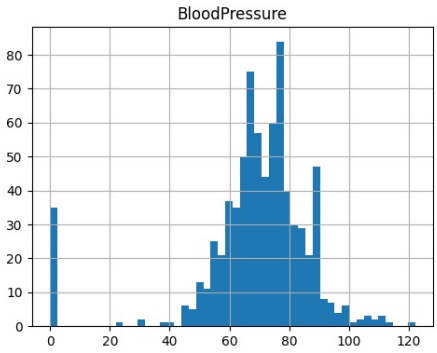
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

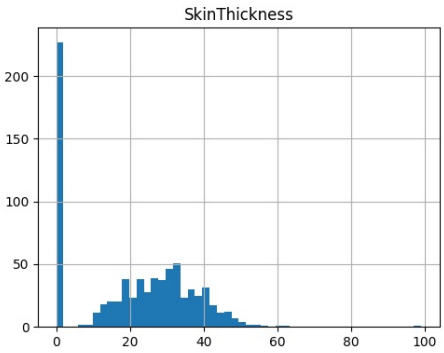
   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

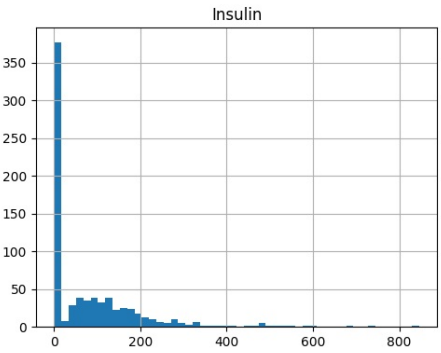
              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```

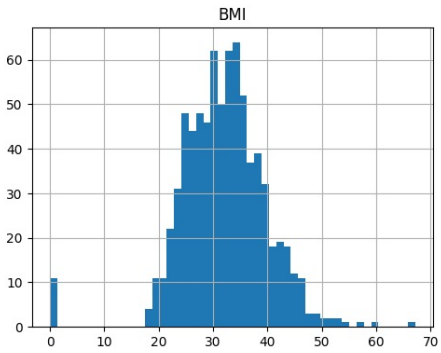

Pregnancies

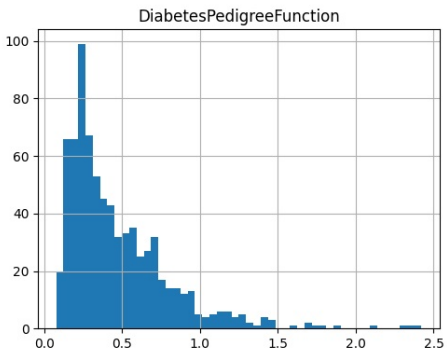

Glucose

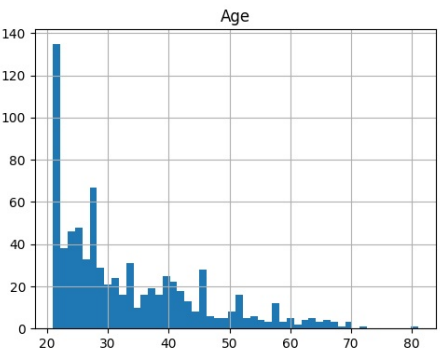

BloodPressure

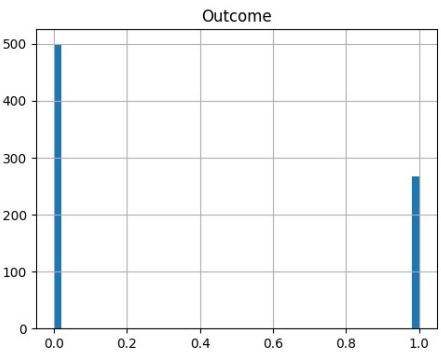

SkinThickness

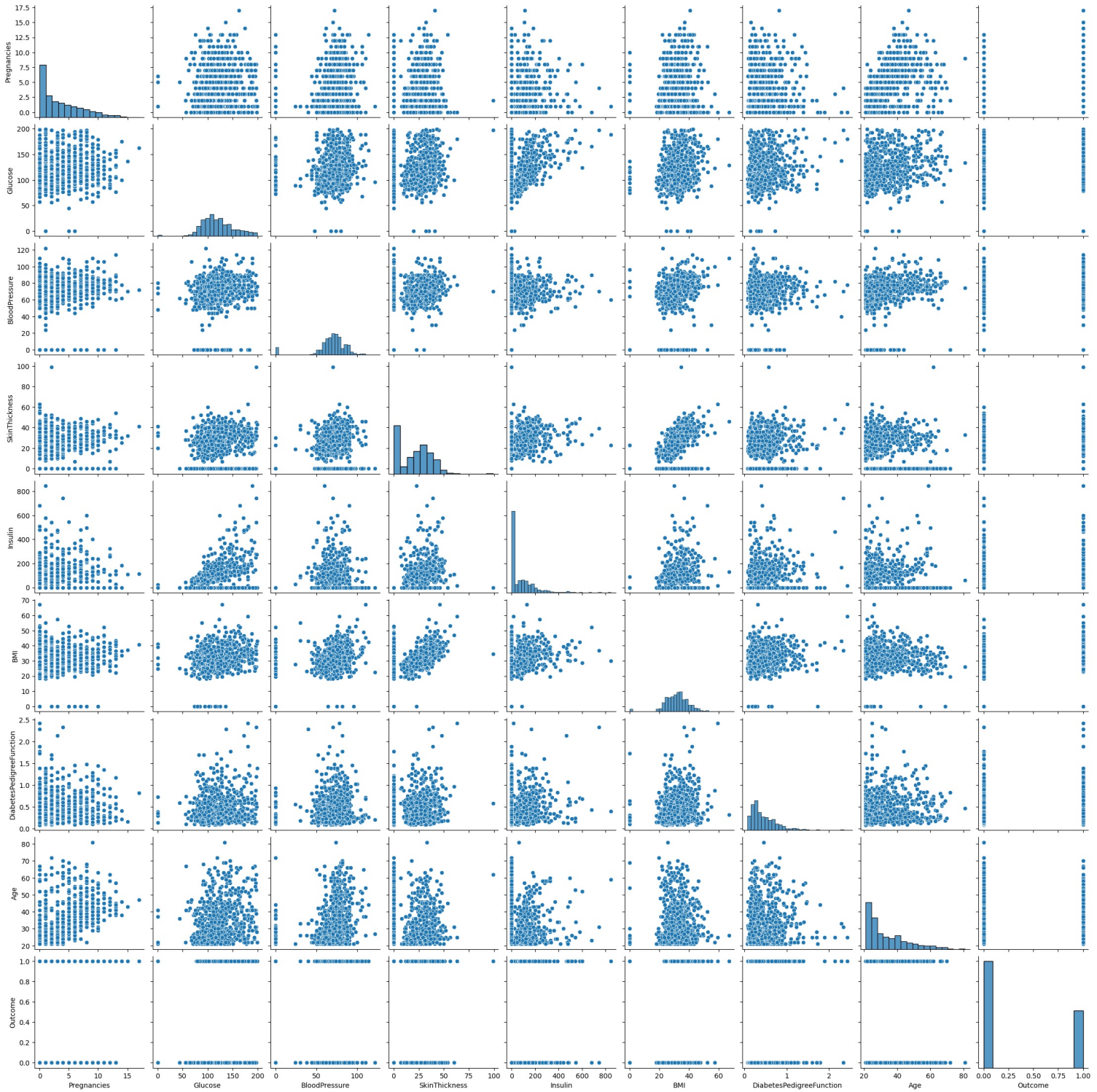

Insulin


BMI


DiabetesPedigreeFunction


Age


Outcome


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

EXPERIMENT NO: 6

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

np.random.seed(42)
dates = pd.date_range(start='2023-01-01', periods=365, freq='D')
visits = np.random.poisson(lam=500, size=365) + np.sin(np.linspace(0, 50, 365)) * 100

visits[50] += 800
visits[180] += 950

data = pd.DataFrame({'Visits': visits}, index=dates)

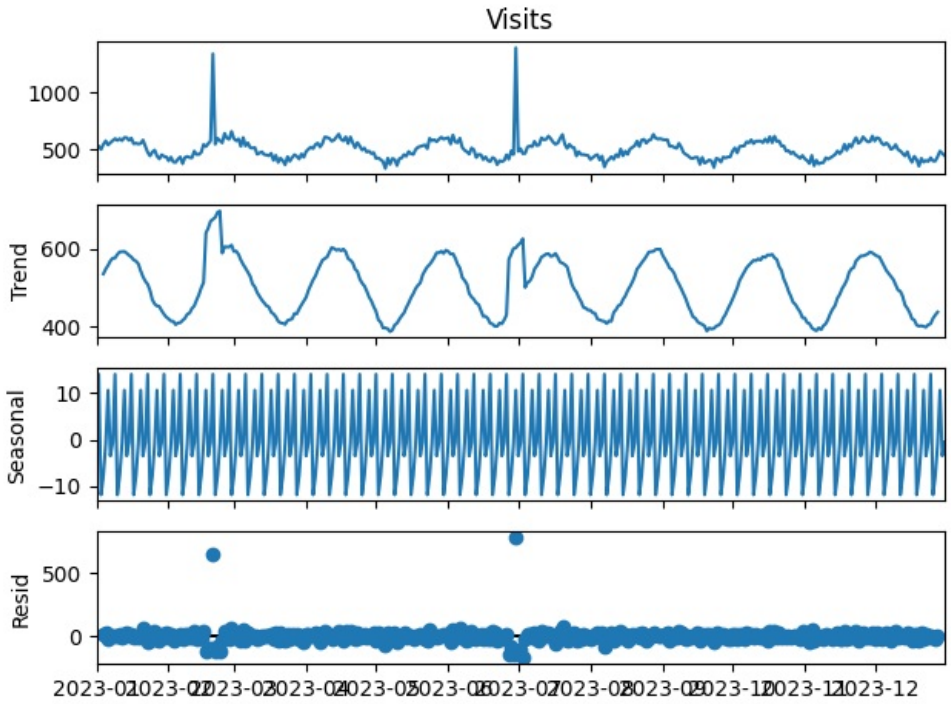
decomposition = seasonal_decompose(data['Visits'], model='additive', period=7)
decomposition.plot()
plt.show()

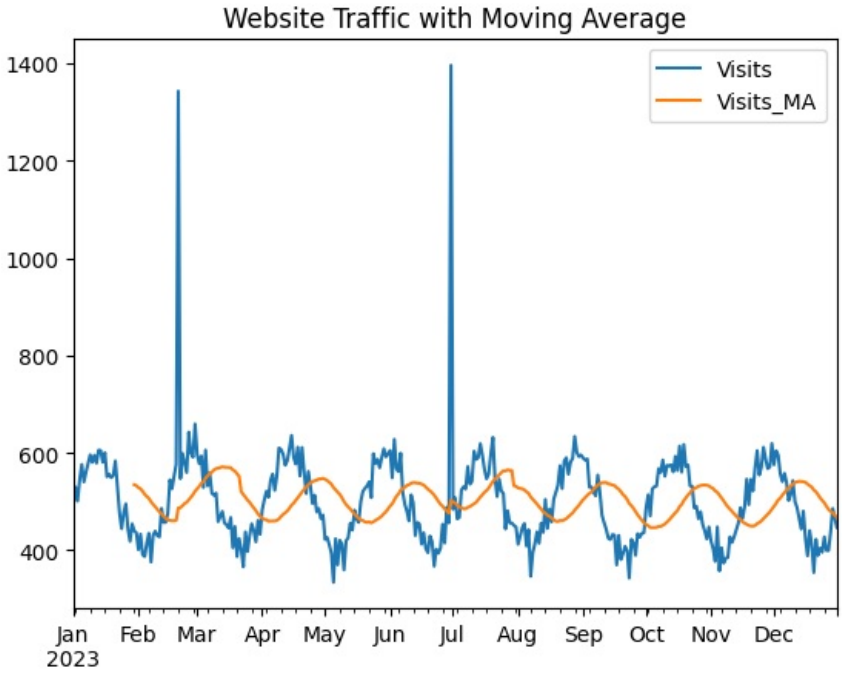
data['Visits_MA'] = data['Visits'].rolling(window=30).mean()
data[['Visits', 'Visits_MA']].plot()
plt.title('Website Traffic with Moving Average')
plt.show()

data['Visits_Z'] = (data['Visits'] - data['Visits'].mean()) / data['Visits'].std()
anomalies = data[data['Visits_Z'].abs() > 3]

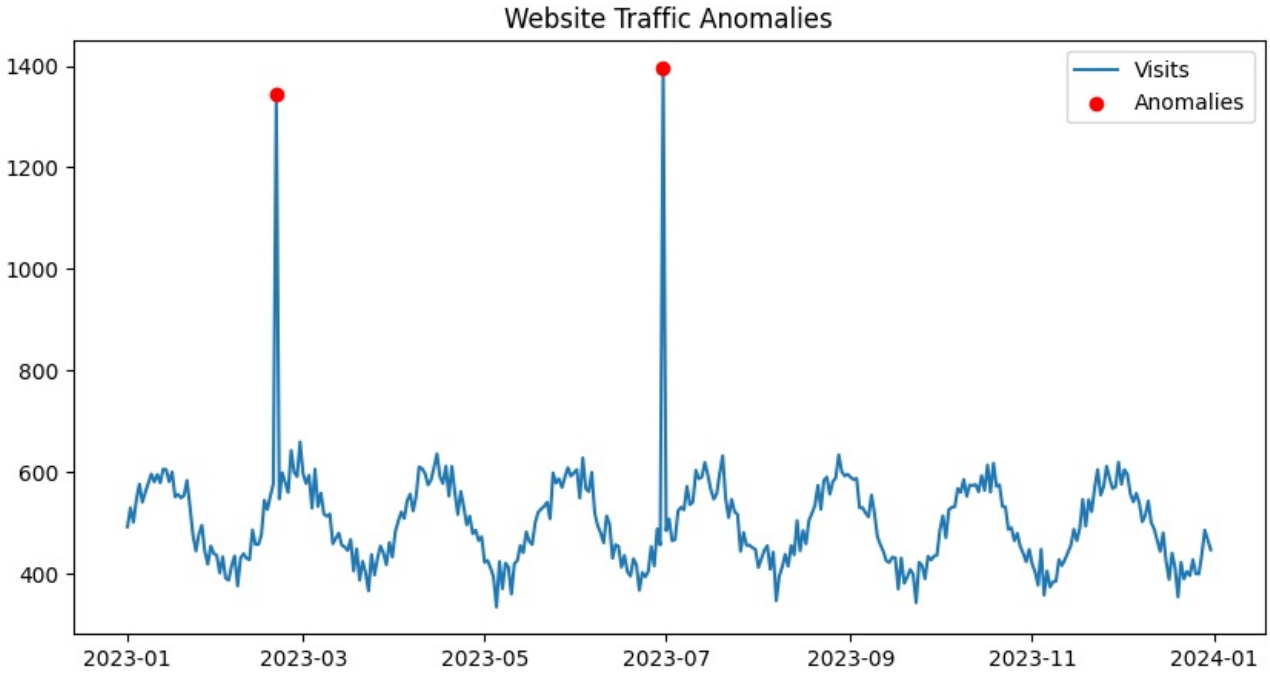
plt.figure(figsize=(10, 5))
plt.plot(data.index, data['Visits'], label='Visits')
plt.scatter(anomalies.index, anomalies['Visits'], color='red', label='Anomalies', zorder = 5)
plt.title('Website Traffic Anomalies')
plt.legend()
```

Output:






```
<matplotlib.legend.Legend at 0x7cce673b9bd0>
```



EXPERIMENT NO: 7

Program:

```
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(population_mean, population_std, population_size)

sample_sizes = [30, 50, 100]
num_samples = 1000

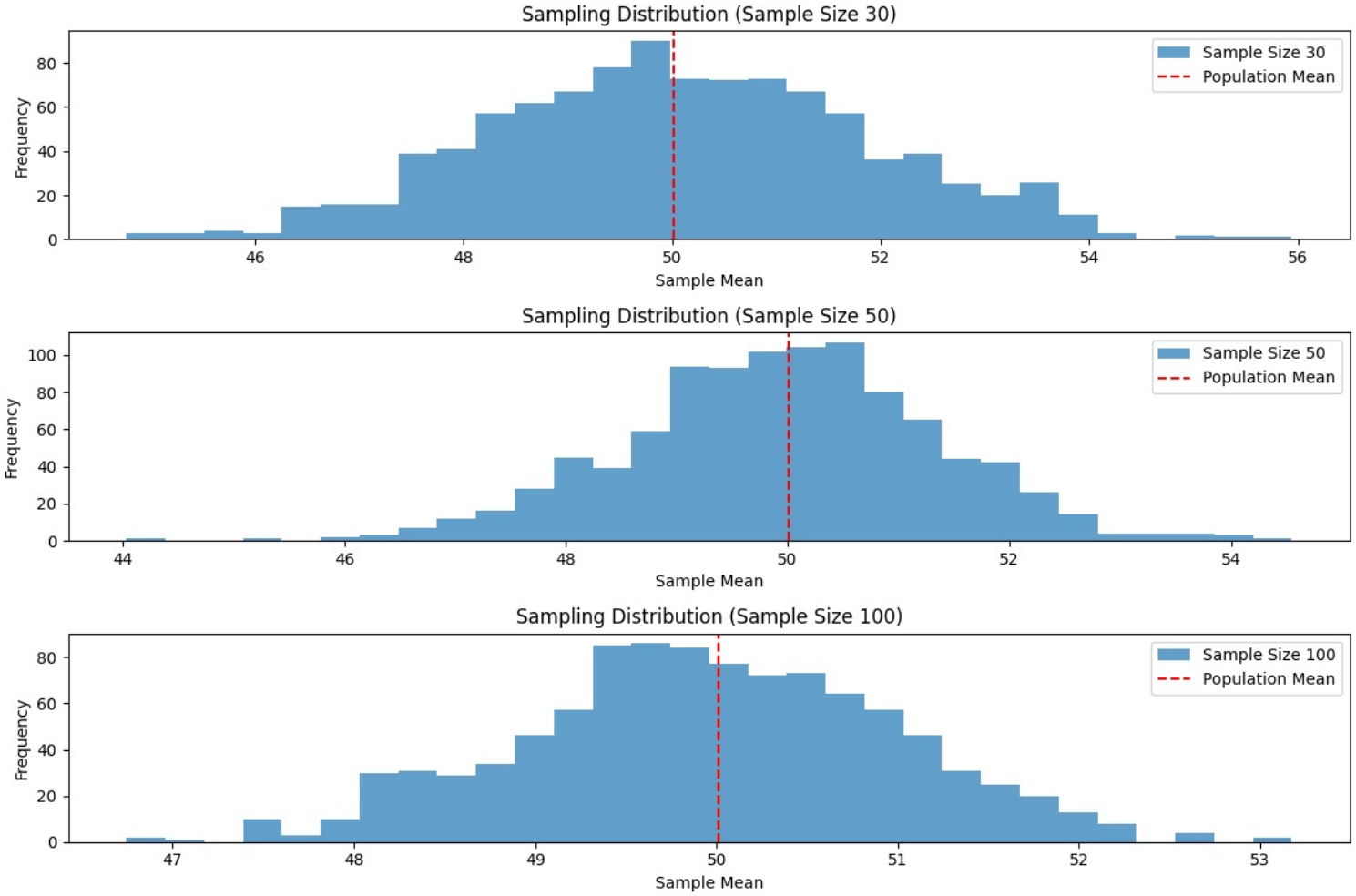
sample_means = {}

for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5, label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```

Output:



EXPERIMENT NO: 8

Program:

```
import numpy as np
import scipy.stats as stats


sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,151, 150, 149, 152, 151,
148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])

population_mean = 150

sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1) # Using sample standard deviation

n = len(sample_data)

z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))


p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic))) # Two-tailed test


print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
   print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")
else:
   print("Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.")
```

Output:

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

EXPERIMENT NO: 9

Program:

```
import numpy as np
import pandas as pd

data = {
    'User ID': range(1, 21),
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male',
               'Female', 'Male', 'Female', 'Male', 'Female',
               'Male', 'Female', 'Male', 'Female', 'Male',
               'Female', 'Male', 'Female', 'Male', 'Female'],
    'Age': [22, 35, 28, 42, 31, 26, 45, 29, 38, 24,
            33, 41, 27, 36, 48, 23, 39, 30, 44, 25],
    'EstimatedSalary': [25000, 65000, 42000, 85000, 55000,
                        32000, 95000, 48000, 72000, 28000,
                        60000, 90000, 35000, 68000, 110000,
                        30000, 78000, 50000, 100000, 40000],
    'Purchased': [0, 1, 0, 1, 1, 0, 1, 0, 1, 0,
                  1, 1, 0, 1, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

print(df)
print(df.head())

features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values

print(features)
print(label)

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features,
        label,
        test_size=0.2,
        random_state=i
    )

    model = LogisticRegression()
    model.fit(x_train, y_train)

    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(
            test_score,
            train_score,
            i
        ))

x_train, x_test, y_train, y_test = train_test_split(
    features,
    label,
    test_size=0.2,
    random_state=42
)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)

print(finalModel.score(x_train, y_train))
print(finalModel.score(x_test, y_test))

from sklearn.metrics import classification_report

print(classification_report(
    label,
    finalModel.predict(features)
))
```

Output:

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0         1    Male   22            25000          0
1         2  Female   35            65000          1
2         3    Male   28            42000          0
3         4  Female   42            85000          1
4         5    Male   31            55000          1
5         6  Female   26            32000          0
6         7    Male   45            95000          1
7         8  Female   29            48000          0
8         9    Male   38            72000          1
9        10  Female   24            28000          0
10       11    Male   33            60000          1
```

```
11       12  Female   41            90000          1
12       13    Male   27            35000          0
13       14  Female   36            68000          1
14       15    Male   48           110000          1
15       16  Female   23            30000          0
16       17    Male   39            78000          1
17       18  Female   30            50000          0
18       19    Male   44           100000          1
19       20  Female   25            40000          0
   User ID  Gender  Age  EstimatedSalary  Purchased
0        1    Male   22            25000          0
1        2  Female   35            65000          1
2        3    Male   28            42000          0
3        4  Female   42            85000          1
4        5    Male   31            55000          1
[[    22  25000]
 [    35  65000]
 [    28  42000]
 [    42  85000]
 [    31  55000]
 [    26  32000]
 [    45  95000]
 [    29  48000]
 [    38  72000]
 [    24  28000]
 [    33  60000]
 [    41  90000]
 [    27  35000]
 [    36  68000]
 [    48 110000]
 [    23  30000]
 [    39  78000]
 [    30  50000]
 [    44 100000]
 [    25  40000]]
[0 1 0 1 1 0 1 0 1 0 1 1 0 1 1 0 1 0 1 0]
1.0
1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         9
           1       1.00      1.00      1.00        11

    accuracy                           1.00        20
   macro avg       1.00      1.00      1.00        20
weighted avg       1.00      1.00      1.00        20
```

EXPERIMENT NO: 10

Program:

```
from google.colab import files
uploaded = files.upload()
```

Output: Choose Files No file selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Mall_Customers - Mall_Customers.xlsx - Mall_Customers - Mall_Customers.csv to Mall_Customers - Mall_Customers.xlsx - Mall_Customers - Mall_Customers (4).csv
```

```
import pandas as pd

df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

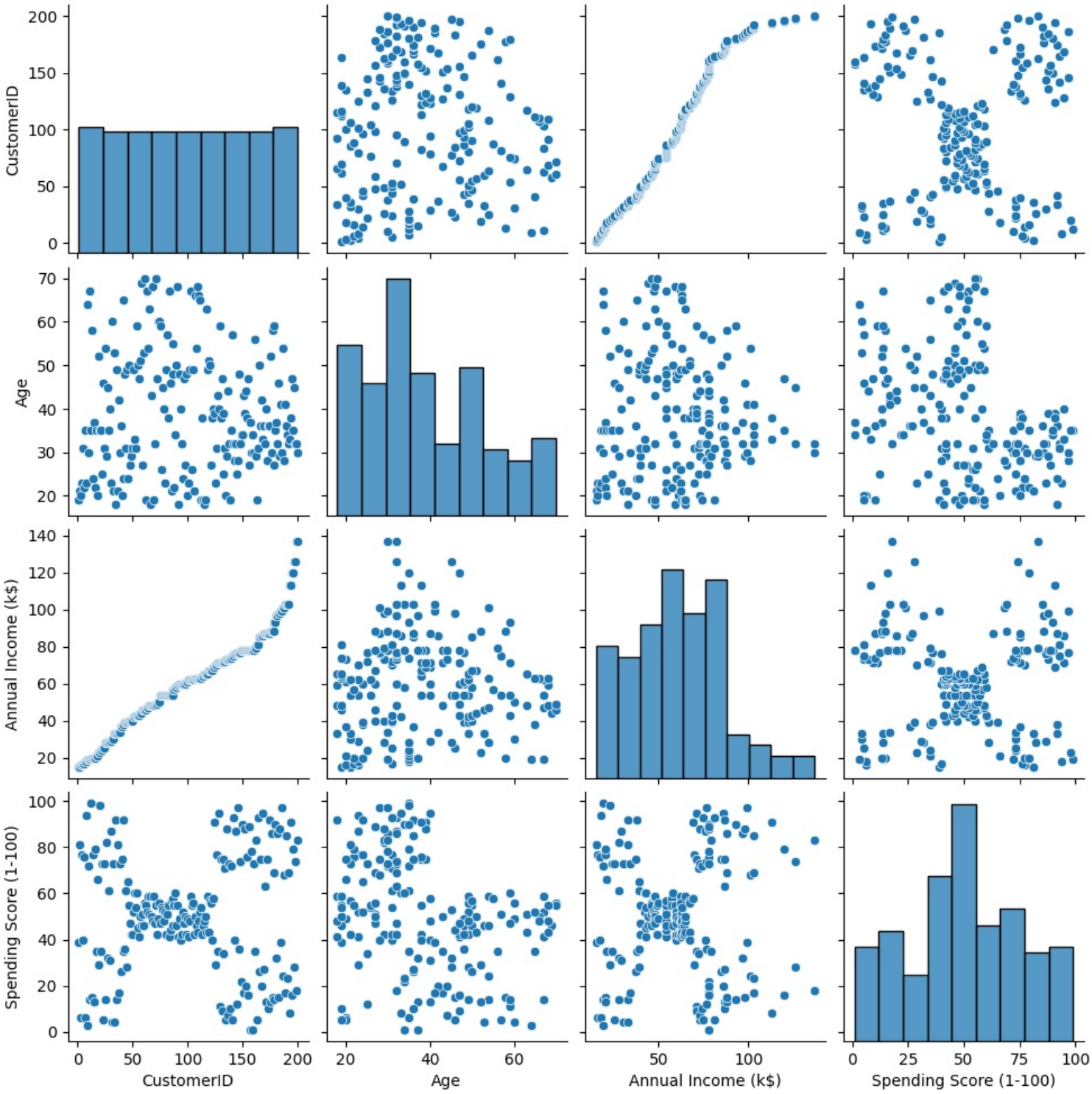
Output:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.pairplot(df)
plt.show()
```

Output:




```
features = df.iloc[:, [3, 4]].values

from sklearn.cluster import KMeans

model = KMeans(n_clusters=5, random_state=42)
model.fit(features)

Final = df.iloc[:, [3, 4]].copy()
Final['label'] = model.predict(features)

Final.head()
```

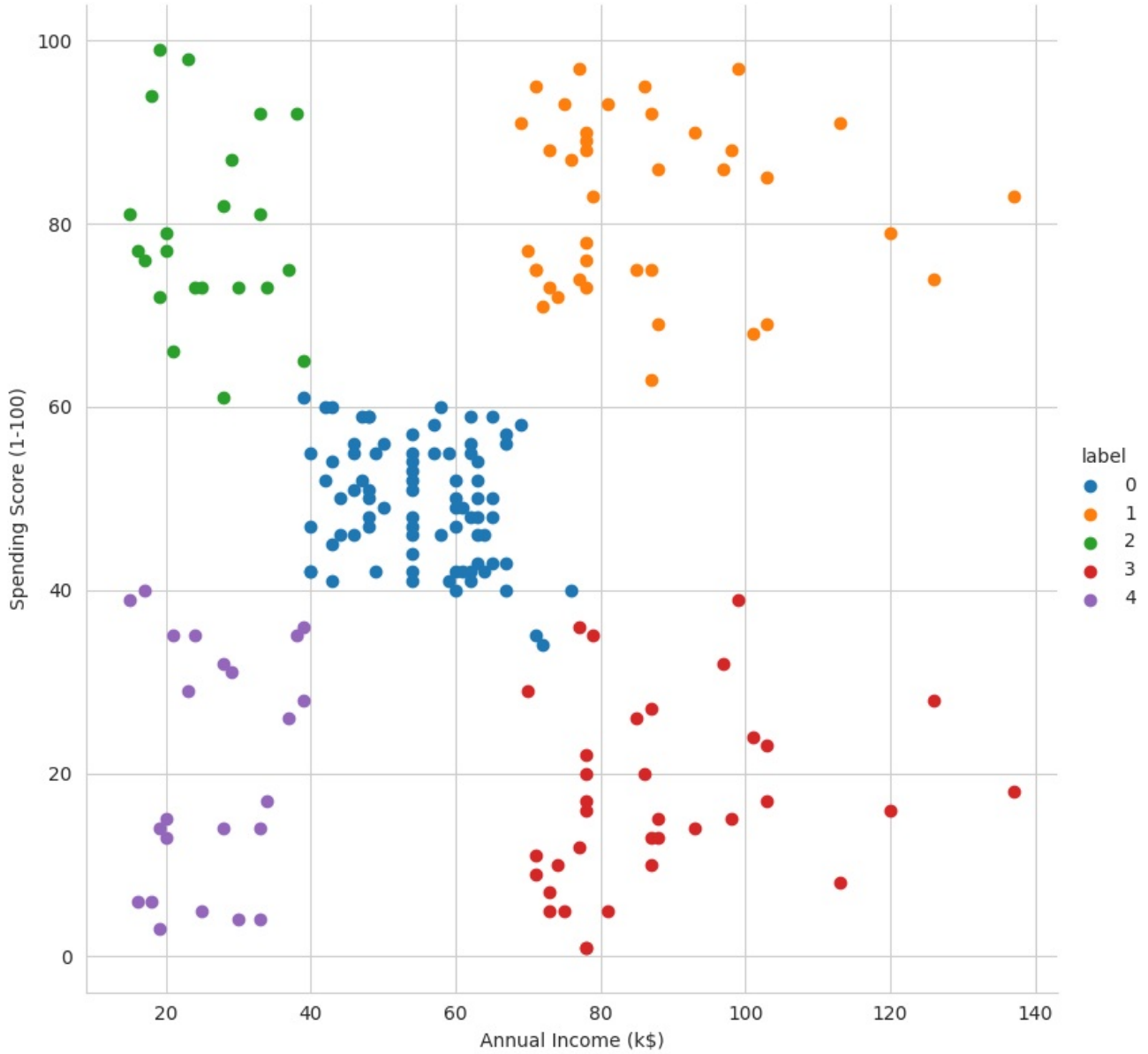
Output:

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4

```
sns.set_style("whitegrid")

sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
    .add_legend()
```

```
plt.show()
```



Output:

EXPERIMENT NO: 11

Experiment to understand pandas library use case in Data Science

Program:

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
print(df)
df.columns=['Empd','Name','Salary']
print(df)
print(df.info())
df=pd.read_csv("50_Startups.csv")
print(df.info())
print(df.head())
print(df.tail())
df=pd.read_csv("employee.csv")
print(df.head())
print(df.tail())
print(df.info())
print(df.salary)
print(type(df.salary))
print(df.salary.mean())
print(df.salary.median())
print(df.salary.mode())
print(df.salary.var())
print(df.salary.std())
print(df.describe())
print(df.describe(include='all'))
empCol=df.columns
print(empCol)
emparray=df.values
print(emparray)
employee_DF=pd.DataFrame(emparray,columns=empCol)
print(employee_DF)
```

Output:

```
  0      1      2
0  1  Smith  50000
1  2  Jones  60000
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
None
   R&D Spend  Administration  Marketing Spend       State     Profit
0   74308.32       180893.92         29300.28    New York   68422.31
1  157853.57       154642.93        304566.74    New York  166448.03
2  126139.12       176832.36        158031.97     Florida  123038.04
```

```
3  106805.48       170801.69        246399.66  California  123228.12
4   42622.70       130716.50        427942.75  California   88386.53
    R&D Spend  Administration  Marketing Spend       State     Profit
45  116065.73       116662.41         31783.56  California   94265.10
46   65198.11       120568.93        292351.77  California   93281.48
47   95409.86       107718.04        243718.96     Florida  111255.33
48   99272.99        53431.58         38422.83     Florida   80461.01
49   46803.90        64565.34        141784.14  California   58761.20
   emp_id                 name  salary
0       1      SREE VARSINI K S    5000
1       2           SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH H G    8000
   emp_id                 name  salary
0       1      SREE VARSINI K S    5000
1       2           SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH H G    8000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp_id  5 non-null      int64
 1   name    5 non-null      object
 2   salary  5 non-null      int64
dtypes: int64(2), object(1)
memory usage: 252.0+ bytes
None
0    5000
1    6000
2    7000
3    5000
```

```
4    8000
Name: salary, dtype: int64
<class 'pandas.core.series.Series'>
6200.0
6000.0
0    5000
Name: salary, dtype: int64
1700000.0
1303.8404810405298
         emp_id       salary
count  5.000000     5.000000
mean   3.000000  6200.000000
std    1.581139  1303.840481
min    1.000000  5000.000000
25%    2.000000  5000.000000
50%    3.000000  6000.000000
75%    4.000000  7000.000000
max    5.000000  8000.000000
          emp_id              name       salary
count   5.000000                 5     5.000000
unique       NaN                 5          NaN
top          NaN  SREE VARSINI K S          NaN
freq         NaN                 1          NaN
mean    3.000000               NaN  6200.000000
std     1.581139               NaN  1303.840481
min     1.000000               NaN  5000.000000
25%     2.000000               NaN  5000.000000
50%     3.000000               NaN  6000.000000
75%     4.000000               NaN  7000.000000
max     5.000000               NaN  8000.000000
Index(['emp_id', 'name', 'salary'], dtype='object')
[[1 'SREE VARSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
```

```
 [5 'SRI AKASH H G' 8000]]
```

	emp_id	name	salary
0	1	SREE VARSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH H G	8000

EXPERIMENT NO: 12

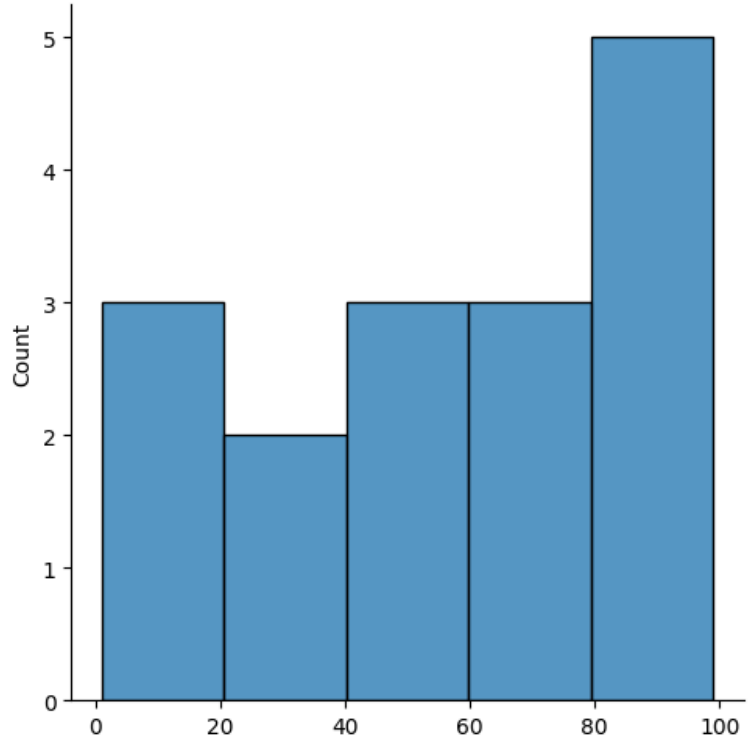
Experiment to detect outliers in a given dataset

Program:

```
import numpy as np
array = np.random.randint(1, 100, 16)
print(array)
print(array.mean())
print(np.percentile(array, 25))
print(np.percentile(array, 50))
print(np.percentile(array, 75))
print(np.percentile(array, 100))
def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur
lr, ur = outDetection(array)
print(lr, ur)
import seaborn as sns
%matplotlib inline
sns.displot(array)
sns.distplot(array)
new_array = array[(array > lr) & (array < ur)]
print(new_array)
sns.displot(new_array)
lr1, ur1 = outDetection(new_array)
print(lr1, ur1)
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
print(final_array)
sns.distplot(final_array)
```

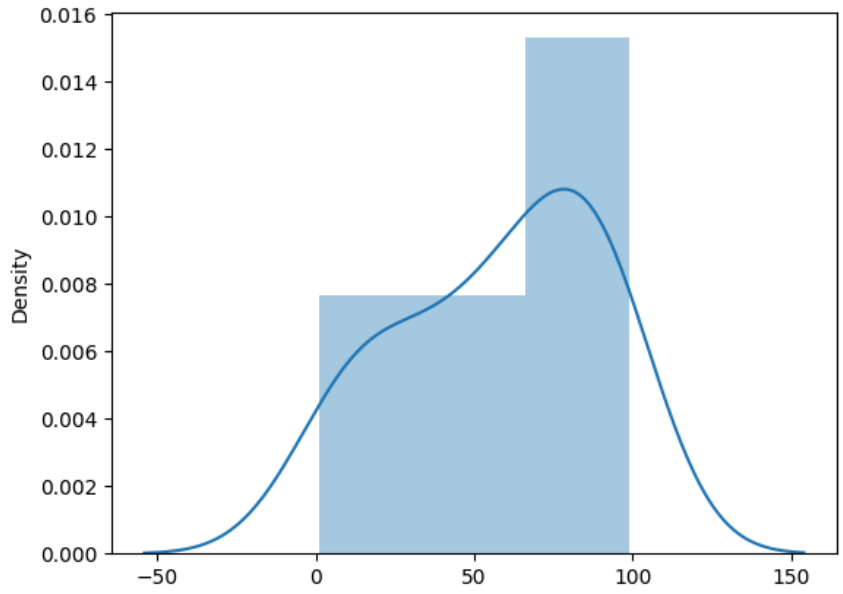
Output:
[99 57 37 75 24 95 78 82 69 90 55 89 1 16 12 46]
57.8125
33.75
63.0
83.75
99.0
-41.25 158.75

<seaborn.axisgrid.FacetGrid at 0x7fae16eacad0>



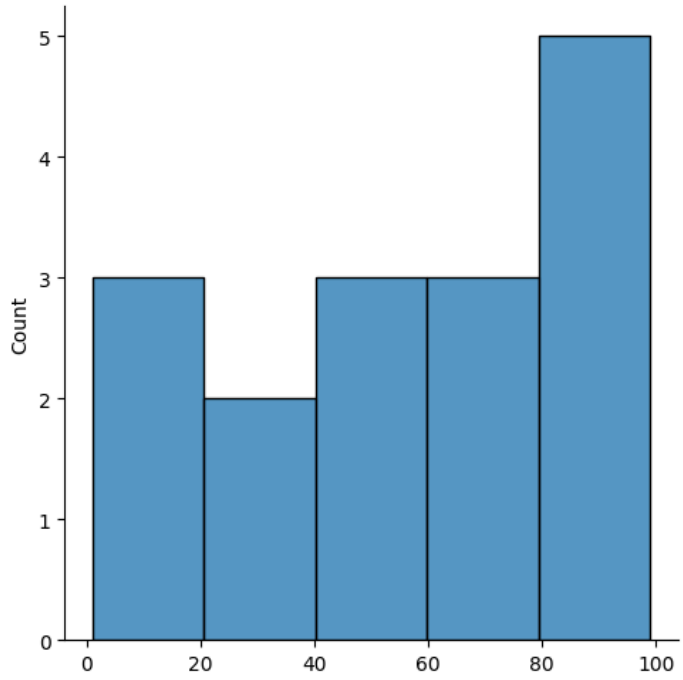
```
sns.distplot(array)
```

```
<Axes: ylabel='Density'>
```




```
[99 57 37 75 24 95 78 82 69 90 55 89  1 16 12 46]
```

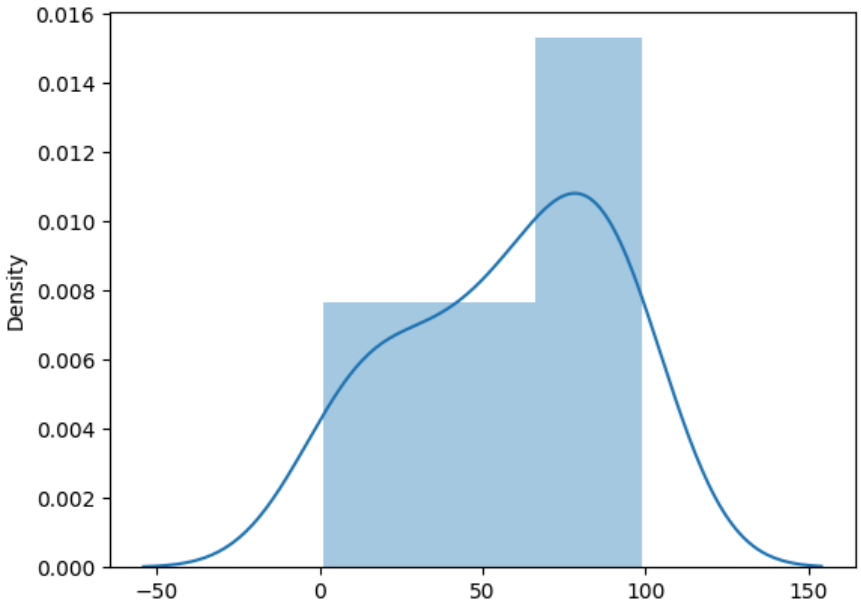
```
<seaborn.axisgrid.FacetGrid at 0x7fae14ac3d90>
```



-41.25 158.75
[99 57 37 75 24 95 78 82 69 90 55 89 1 16 12 46]

```
sns.distplot(final_array)
```

<Axes: ylabel='Density'>



EXPERIMENT NO: 13

Experiment to handle misssing values and inapproriate data in a given dataset

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv("Hotel_Dataset.csv")
print(df)
print(df.duplicated())
print(df.info())
df.drop_duplicates(inplace=True)
print(df)
print(len(df))
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
print(index)
print(df)
df.drop(['Age_Group.1'],axis=1,inplace=True)
print(df)
df.CustomerID.loc[df.CustomerID<0]=np.nan
df.Bill.loc[df.Bill<0]=np.nan
df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
print(df)
df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
print(df)
print(df.Age_Group.unique())
print(df.Hotel.unique())
df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
print(df.FoodPreference.unique)
df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
df.FoodPreference.replace(['non-Veg'],'Non-Veg',inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
print(df)
```

Output:

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0           1     20-25           4       Ibis           veg  1300
1           2     30-35           5  LemonTree        Non-Veg  2000
2           3     25-30           6     RedFox           Veg  1322
3           4     20-25          -1  LemonTree           Veg  1234
4           5       35+           3       Ibis     Vegetarian   989
5           6       35+           3       Ibys        Non-Veg  1909
6           7       35+           4     RedFox     Vegetarian  1000
7           8     20-25           7  LemonTree           Veg  2999
8           9     25-30           2       Ibis        Non-Veg  3456
9           9     25-30           2       Ibis        Non-Veg  3456
10         10     30-35           5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
9         3          -99999       25-30
10        4           87777       30-35
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9     True
10   False
dtype: bool
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
```

```
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
None
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis           veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox           Veg  1322
3            4     20-25           -1  LemonTree           Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree           Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
10        4           87777       30-35
10
[0 1 2 3 4 5 6 7 8 9]
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
```

0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

	EstimatedSalary	Age_Group.1
0	40000	20-25
1	59000	30-35
2	30000	25-30
3	120000	20-25
4	45000	35+
5	122220	35+
6	21122	35+
7	345673	20-25
8	-99999	25-30
9	87777	30-35

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	\
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

	EstimatedSalary
0	40000
1	59000
2	30000
3	120000
4	45000
5	122220

```
6          21122
7         345673
8         -99999
9          87777
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25           4       Ibis           veg  1300.0
1         2.0     30-35           5  LemonTree        Non-Veg  2000.0
2         3.0     25-30           6     RedFox           Veg  1322.0
3         4.0     20-25          -1  LemonTree           Veg  1234.0
4         5.0       35+           3       Ibis     Vegetarian   989.0
5         6.0       35+           3       Ibys        Non-Veg  1909.0
6         7.0       35+           4     RedFox     Vegetarian  1000.0
7         8.0     20-25           7  LemonTree           Veg  2999.0
8         9.0     25-30           2       Ibis        Non-Veg  3456.0
9        10.0     30-35           5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0        2          40000.0
1        3          59000.0
2        2          30000.0
3        2         120000.0
4        2          45000.0
5        2         122220.0
6       -1          21122.0
7      -10         345673.0
8        3              NaN
9        4          87777.0
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25           4       Ibis           veg  1300.0
1         2.0     30-35           5  LemonTree        Non-Veg  2000.0
2         3.0     25-30           6     RedFox           Veg  1322.0
3         4.0     20-25          -1  LemonTree           Veg  1234.0
4         5.0       35+           3       Ibis     Vegetarian   989.0
5         6.0       35+           3       Ibys        Non-Veg  1909.0
6         7.0       35+           4     RedFox     Vegetarian  1000.0
7         8.0     20-25           7  LemonTree           Veg  2999.0
8         9.0     25-30           2       Ibis        Non-Veg  3456.0
9        10.0     30-35           5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
```

```
1     3.0         59000.0
2     2.0         30000.0
3     2.0        120000.0
4     2.0         45000.0
5     2.0        122220.0
6     NaN         21122.0
7     NaN        345673.0
8     3.0             NaN
9     4.0         87777.0
['20-25' '30-35' '25-30' '35+']
['Ibis' 'LemonTree' 'RedFox' 'Ibys']
<bound method Series.unique of 0          veg
1      Non-Veg
2          Veg
3          Veg
4    Vegetarian
5      Non-Veg
6    Vegetarian
7          Veg
8      Non-Veg
9      non-Veg
Name: FoodPreference, dtype: object>
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree       Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis       Non-Veg  1909.0
6         7.0       35+            4     RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis       Non-Veg  3456.0
9        10.0     30-35            5     RedFox       Non-Veg  1801.0

   NoOfPax  EstimatedSalary
0      2.0         40000.0
1      3.0         59000.0
2      2.0         30000.0
3      2.0        120000.0
4      2.0         45000.0
5      2.0        122220.0
```

6	2.0	21122.0
7	2.0	345673.0
8	3.0	96755.0
9	4.0	87777.0

EXPERIMENT NO: 14

Experiment to understand feature scaling

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('pre-process_datasample.csv')
print(df)
print(df.head())
df.Country.fillna(df.Country.mode()[0],inplace=True)
features=df.iloc[:,:-1].values
print(features)
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
Salary.fit(features[:,[2]])
SimpleImputer()
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
print(features)
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
print(Country)
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
print(final_set)
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
print(feat_standard_scaler)
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
print(feat_minmax_scaler)
```

Output:

```
  Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
  Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
```

```
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]]
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
```

```
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00  0.00000000e+00
  -1.07356980e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  1.34013983e+00
    1.38753832e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.63077256e+00
    1.75214693e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -2.58340208e-01
    2.93712492e-01]]
[[1.         0.         0.         0.73913043 0.68571429]
 [0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.13043478 0.17142857]
 [0.         0.         1.         0.47826087 0.37142857]
 [0.         1.         0.         0.56521739 0.45079365]
 [1.         0.         0.         0.34782609 0.28571429]
 [0.         0.         1.         0.51207729 0.11428571]
 [1.         0.         0.         0.91304348 0.88571429]
 [0.         1.         0.         1.         1.        ]
 [1.         0.         0.         0.43478261 0.54285714]]
```

EXPERIMENT No: 15

Experiment to understand the data preprocessing in data science

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv("pre-process_datasample.csv")
print(df)
print(df.info())
print(df.Country.mode())
print(df.Country.mode()[0])
print(type(df.Country.mode()))
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
print(df)
print(pd.get_dummies(df.Country))
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],
axis=1)
print(updated_dataset)
print(df.info())
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
print(updated_dataset)
```

Output:

```
  Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
```

```
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
None
0    France
Name: Country, dtype: object
France
<class 'pandas.core.series.Series'>
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8   False     True  False
9    True    False  False
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
```

```
8   False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0       Yes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
None
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8   False     True  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1
```

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
```

EXPERIMENT NO: 16

Experiment to understand EDA - Quantitative and Qualitative analysis

Program:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
print(tips.head())
sns.displot(tips.total_bill,kde=True)
sns.displot(tips.total_bill,kde=False)
sns.jointplot(x=tips.tip,y=tips.total_bill)
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
sns.pairplot(tips)
print(tips.time.value_counts())
sns.pairplot(tips,hue='time')
sns.pairplot(tips, hue='day')
plt.show()
plt.close()
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()
plt.close()plt.figure()
sns.boxplot(y=tips.total_bill)
plt.show()
plt.close()
plt.figure()
sns.boxplot(y=tips.tip)
plt.show()
plt.close()

plt.figure()
sns.countplot(x=tips.day)
plt.show()
plt.close()
```

```
plt.figure()
sns.countplot(x=tips.sex)
plt.show()
plt.close()

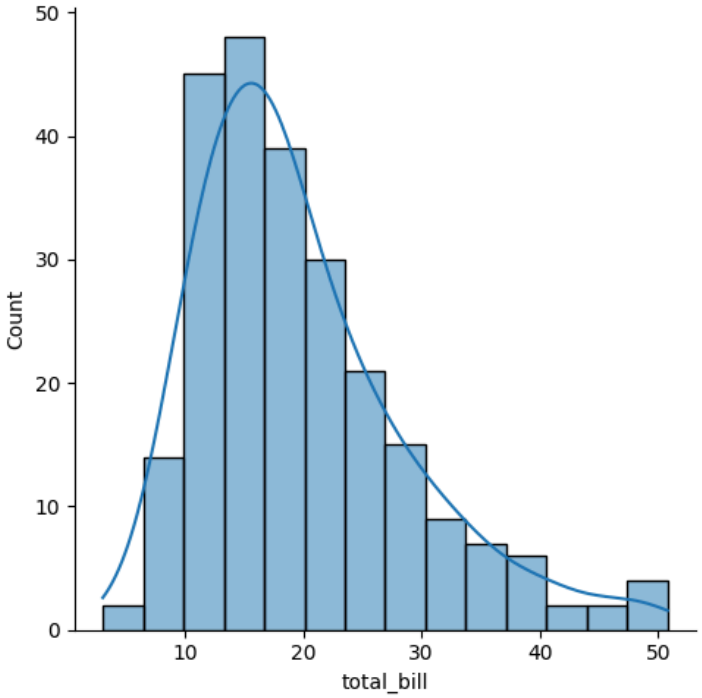
plt.figure()
tips.sex.value_counts().plot(kind='pie')
plt.show()
plt.close()

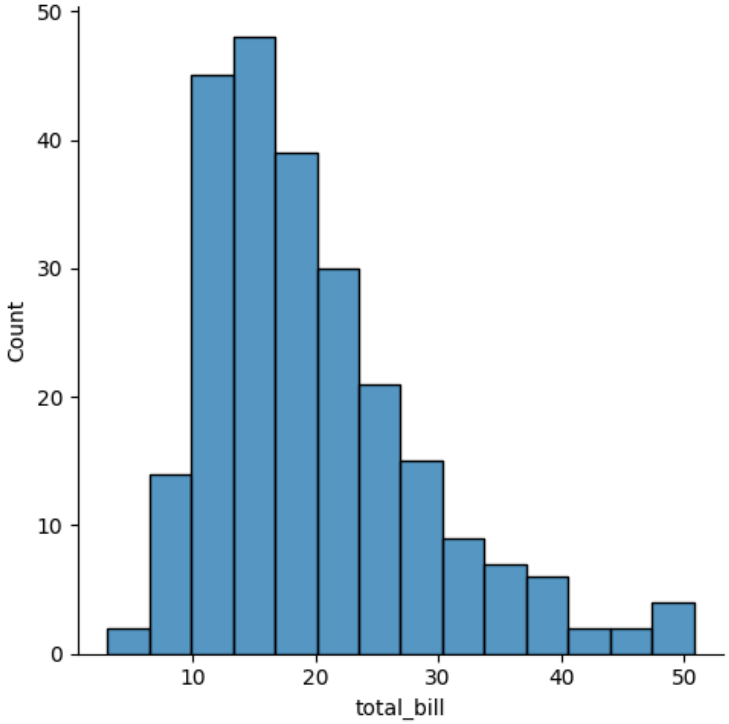
plt.figure()
tips.sex.value_counts().plot(kind='bar')
plt.show()
plt.close()

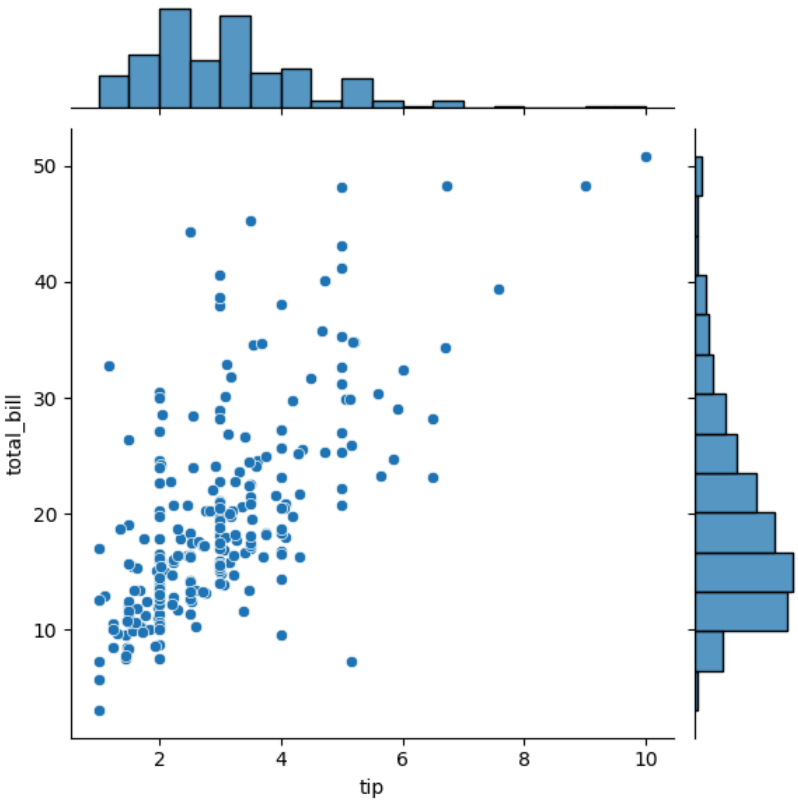
plt.figure()
sns.countplot(x=tips[tips.time == 'Dinner']['day'])
plt.show()
plt.close()
```

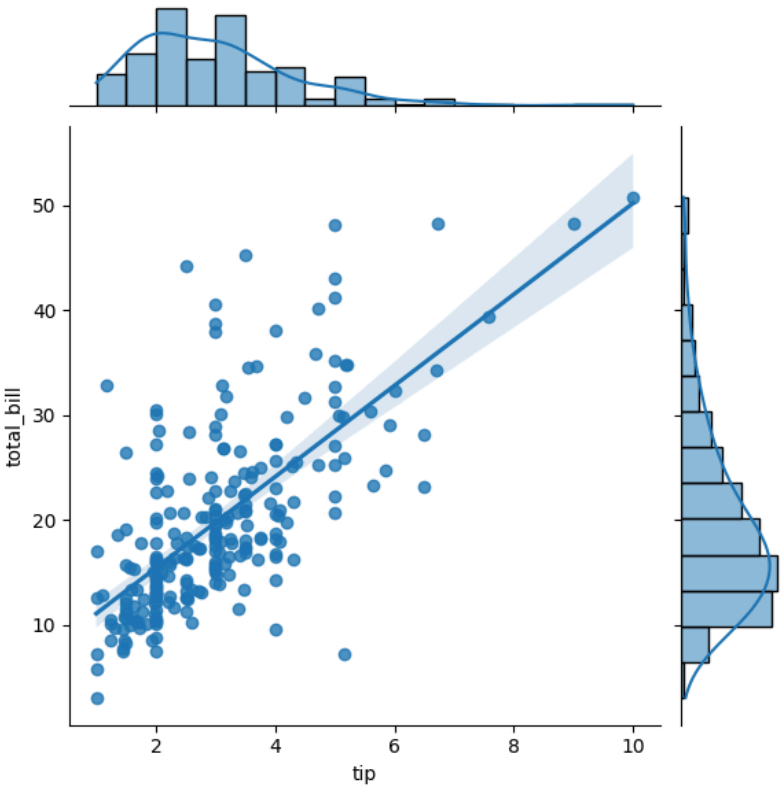
Output:

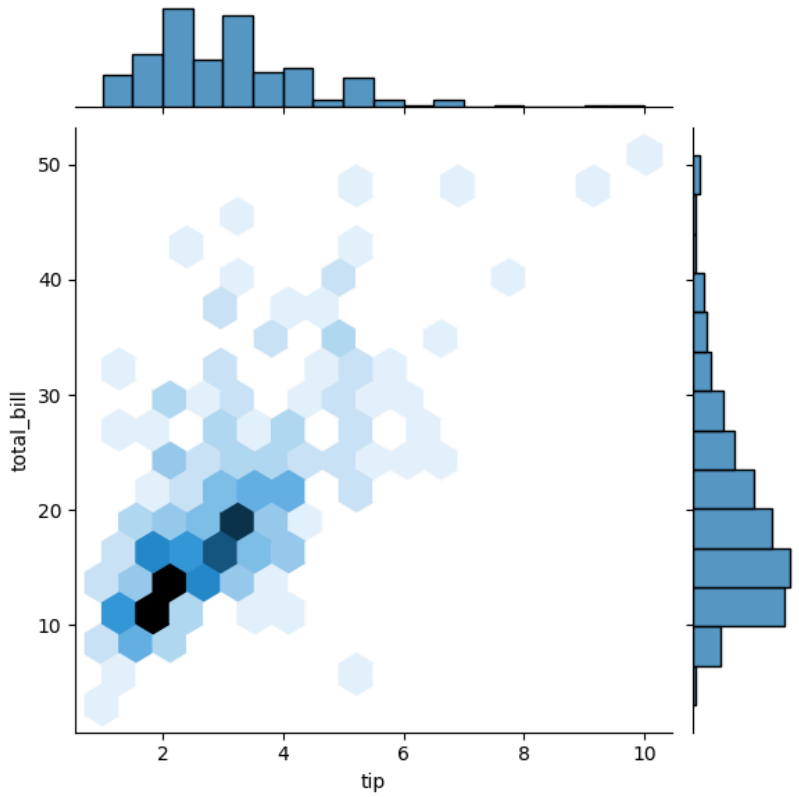
```
  total_bill   tip     sex smoker  day    time  size
0      16.99  1.01  Female     No  Sun  Dinner     2
1      10.34  1.66    Male     No  Sun  Dinner     3
2      21.01  3.50    Male     No  Sun  Dinner     3
3      23.68  3.31    Male     No  Sun  Dinner     2
4      24.59  3.61  Female     No  Sun  Dinner     4
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

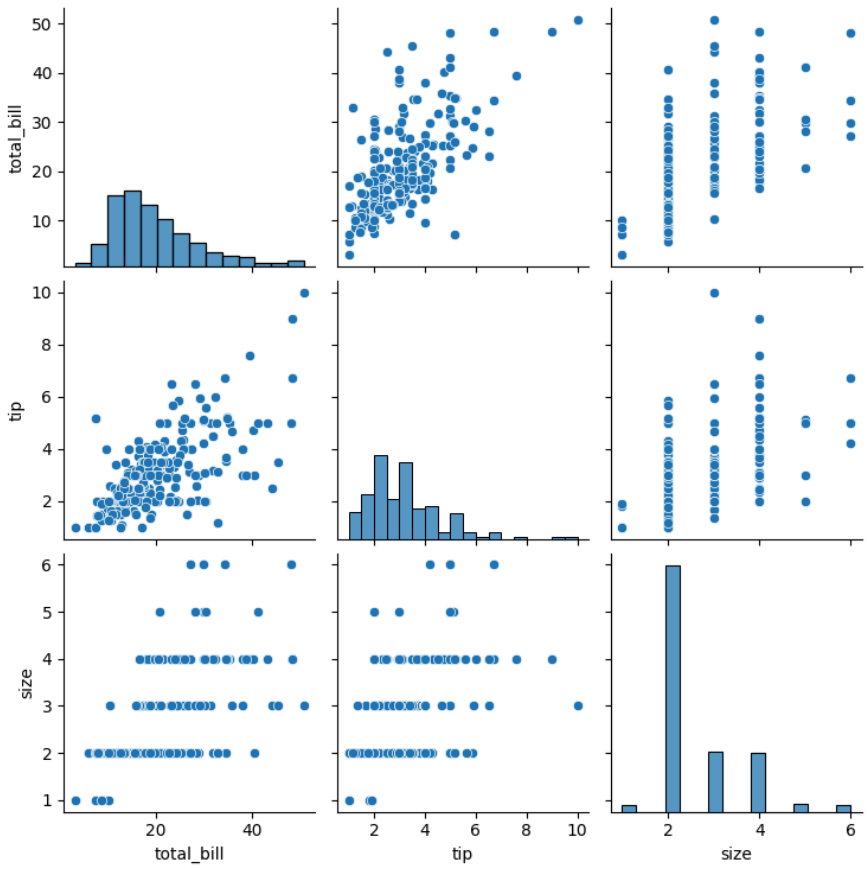

Count
total_bill
0
10
20
30
40
50

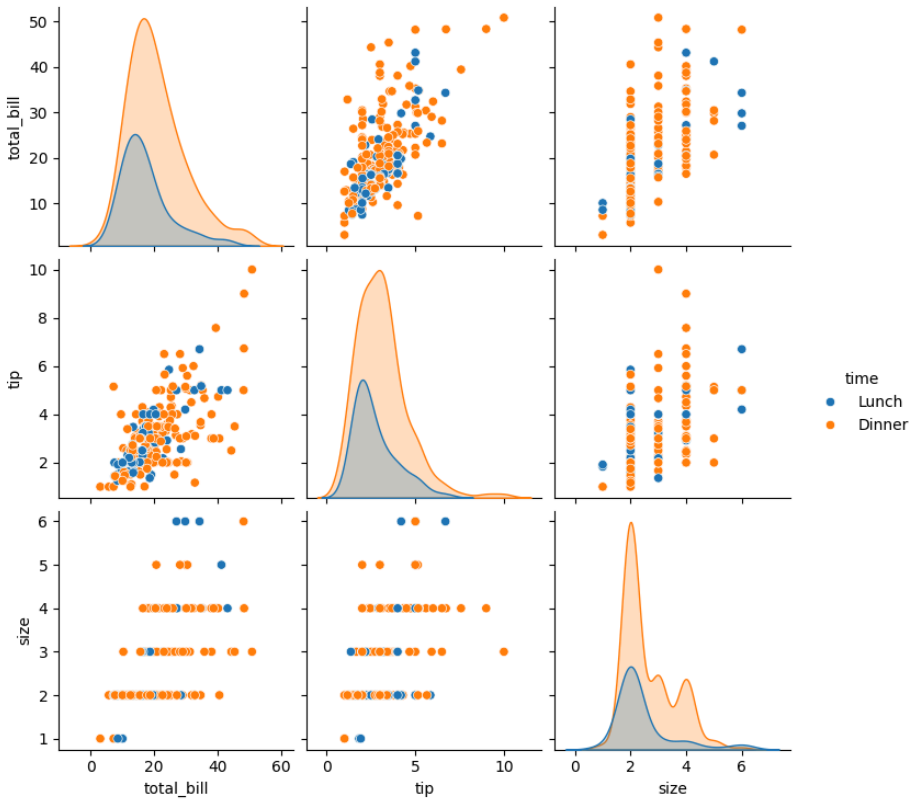

Count
total_bill
0
10
20
30
40
50

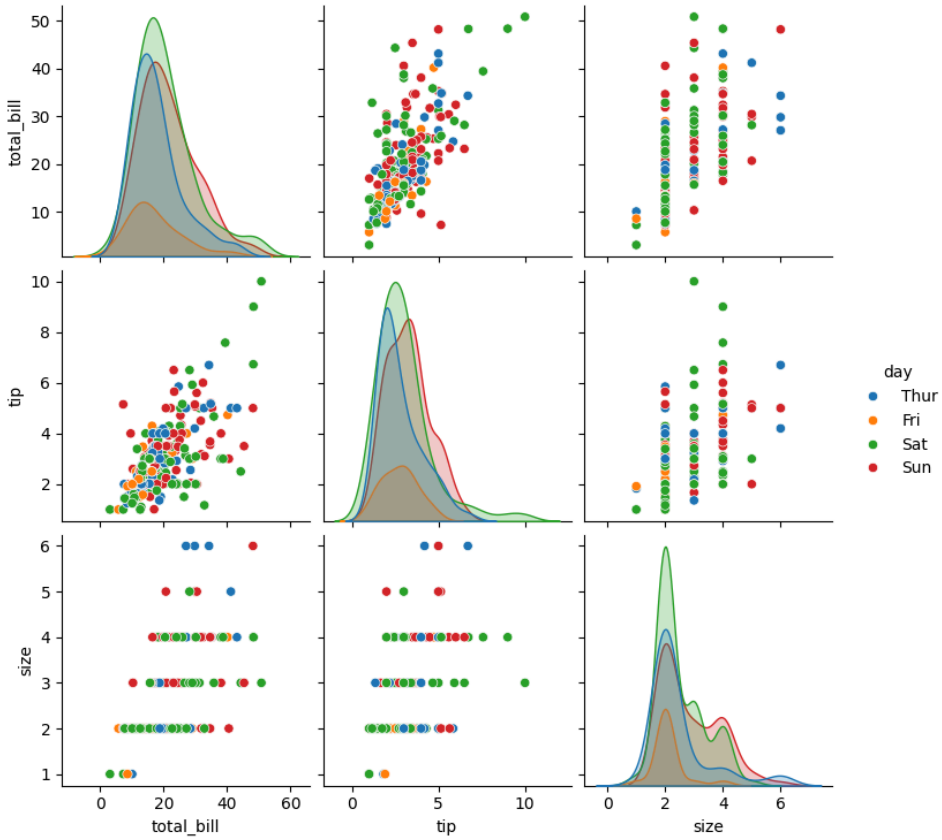

total_bill
tip
50
40
30
20
10
2
4
6
8
10

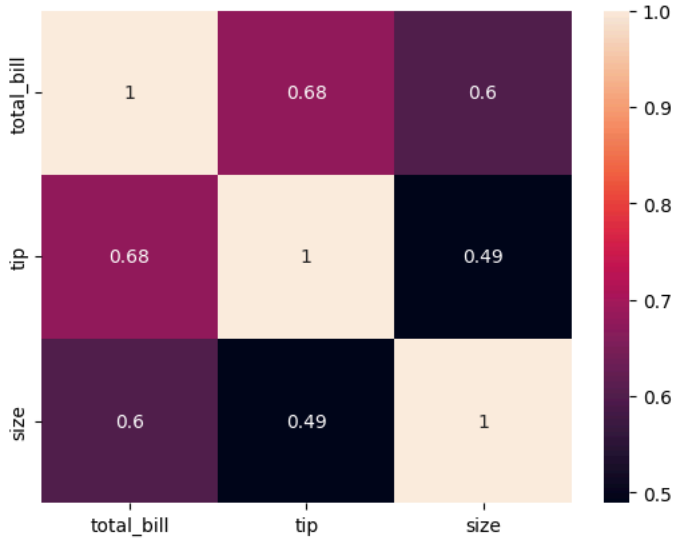

total_bill
tip
50
40
30
20
10
2
4
6
8
10

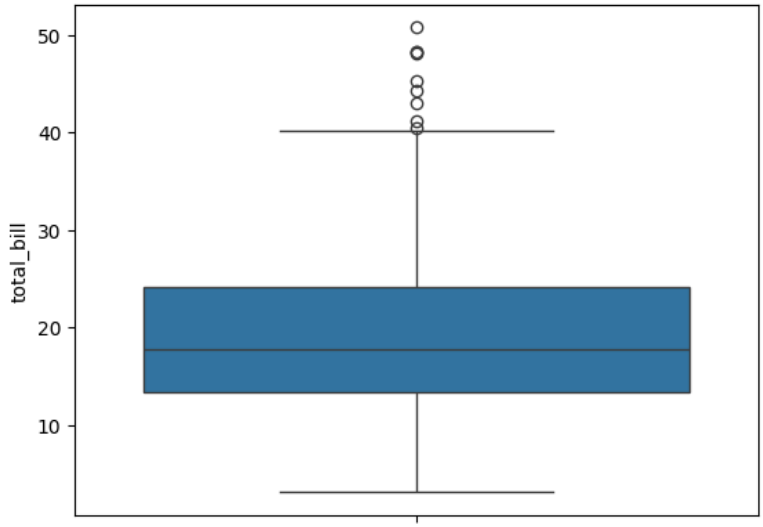

total_bill
tip
50
40
30
20
10
2
4
6
8
10

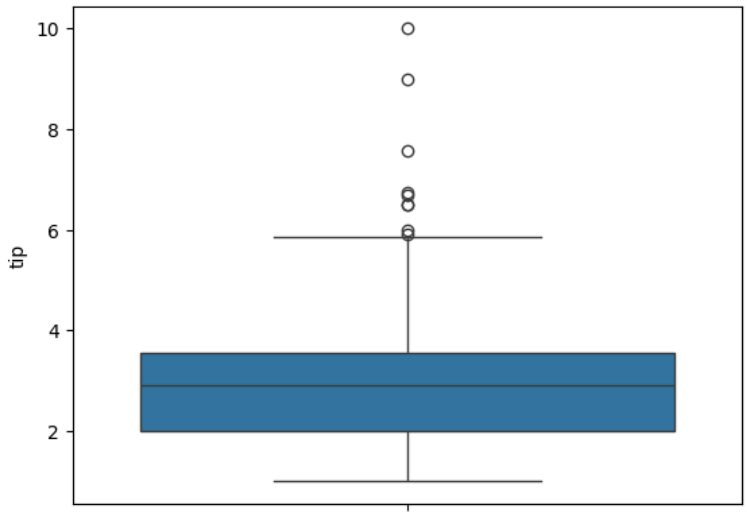

total_bill
tip
size
50
40
30
20
10
10
8
6
4
2
6
5
4
3
2
1
20
40
2
4
6
8
10
2
4
6

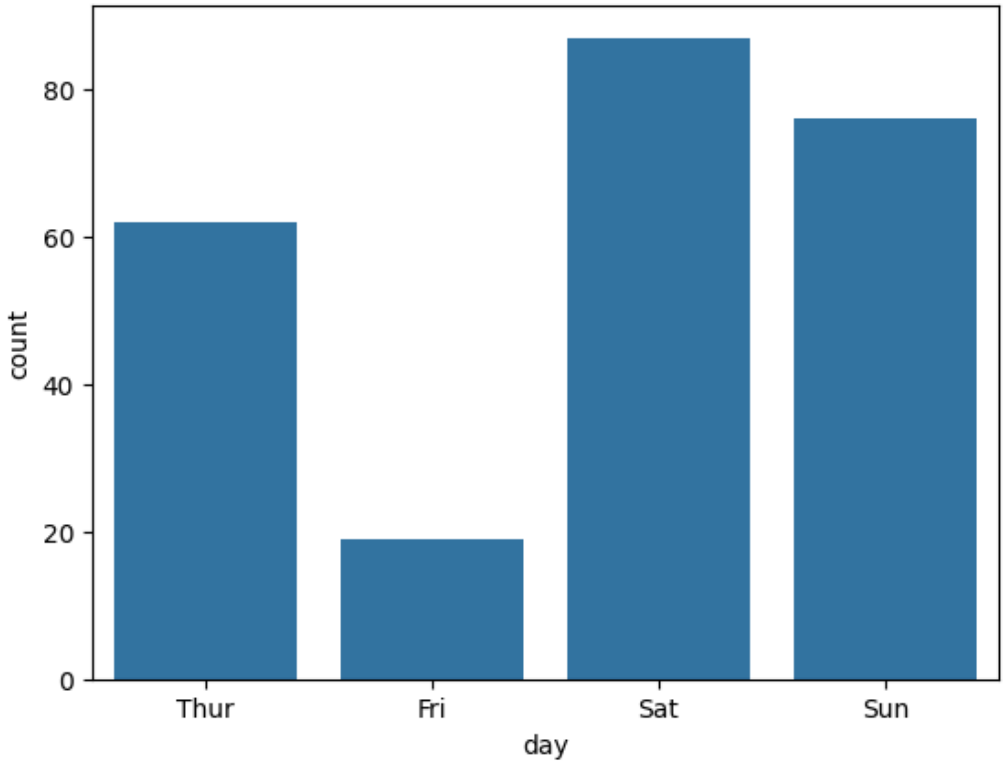

total_bill
tip
size
time
Lunch
Dinner

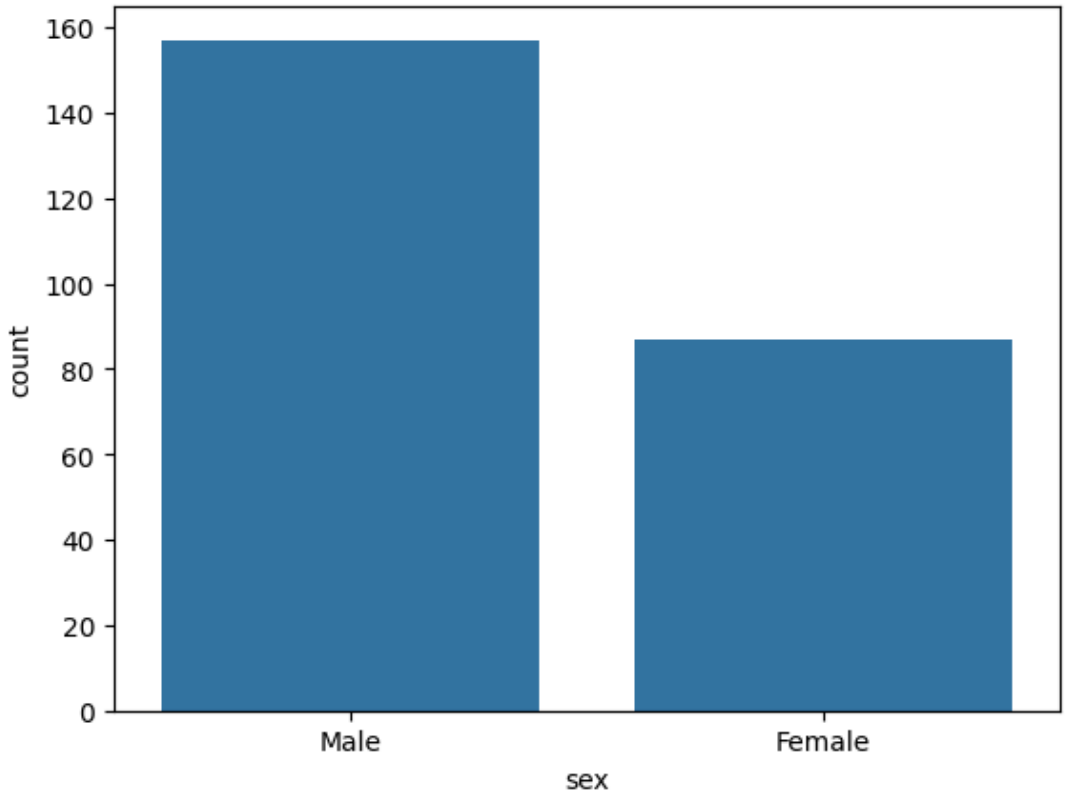

total_bill
tip
size
day
Thur
Fri
Sat
Sun

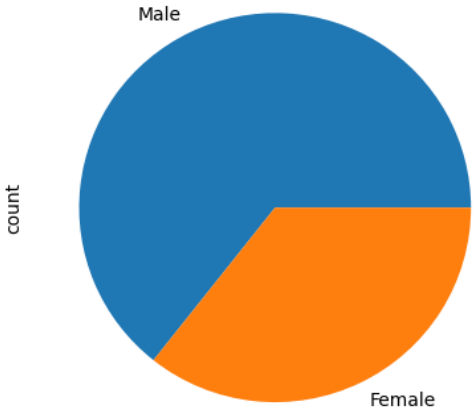

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5

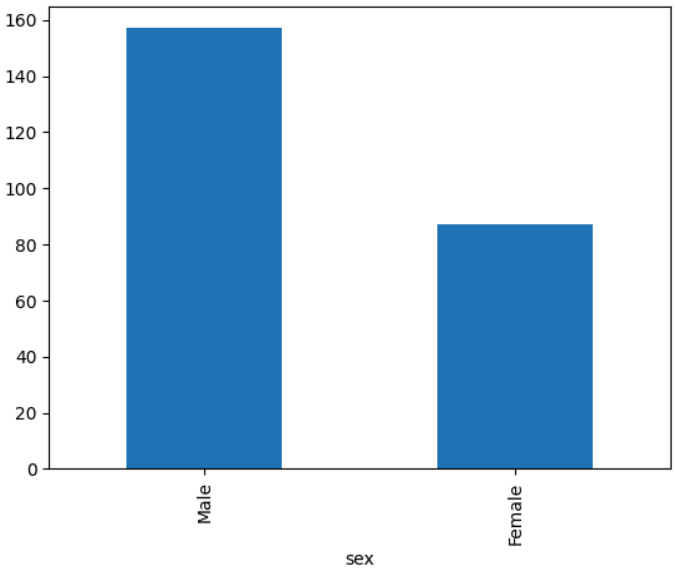

total_bill
50
40
30
20
10

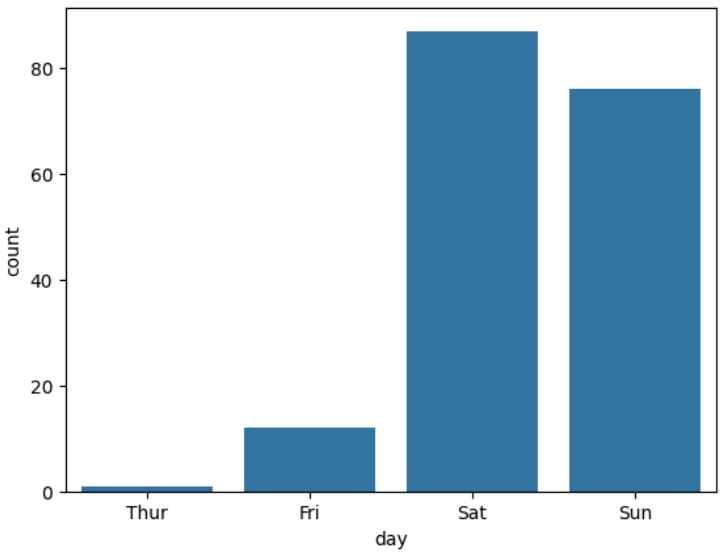

tip
10
8
6
4
2


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day


160
140
120
100
80
60
40
20
0
count
Male
Female
sex


Male
count
Female


160
140
120
100
80
60
40
20
0
Male
Female
sex


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day

EXPERIMENT No: 17

Experiment to understand Linear Regression for a given data set.

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
print(df.info())
df.dropna(inplace=True)
print(df.info())
print(df.describe())
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features, label,
test_size=0.2,random_state=42)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
print(model.score(x_train,y_train))
print(model.score(x_test,y_test))
print(model.coef_)
print(model.intercept_)
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}:
".format(yr_of_exp,Salary))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
None
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
0.9645401573418146
0.9024461774180497
[[9423.81532303]]
[25321.58301178]
Enter Years of Experience: 44
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

EXPERIMENT NO: 18

Experiment to understand KNN algorithm for a given dataset

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Iris.csv')
print(df.info())
print(df.variety.value_counts())
print(df.head())
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,ran
dom_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
print(confusion_matrix(label,model_KNN.predict(features)))
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
```

```
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
0.9666666666666667
1.0
[[50  0  0]
 [ 0 47  3]
 [ 0  1 49]]
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

EXPERIMENT NO: 19

Experiment to understand Logistic Regression for a given dataset.

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
print(df)
print(df.head())
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
print(features)
print(label)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1,401):

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.
2,random_state=42)
  model=LogisticRegression()
  model.fit(x_train,y_train)
  train_score=model.score(x_train,y_train)
  test_score=model.score(x_test,y_test)
  if test_score>train_score:
      print("Test {} Train {} Random State
{}".format(test_score,train_score,i))
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.
2,random_state=42)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

Output:

```
      User ID  Gender  Age  EstimatedSalary  Purchased
0    15624510    Male   19            19000          0
1    15810944    Male   35            20000          0
2    15668575  Female   26            43000          0
3    15603246  Female   27            57000          0
4    15804002    Male   19            76000          0
..        ...     ...  ...              ...        ...
395  15691863  Female   46            41000          1
396  15706071    Male   51            23000          1
397  15654296  Female   50            20000          1
398  15755018    Male   36            33000          0
399  15594041  Female   49            36000          1

[400 rows x 5 columns]
   User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
[[    19  19000]
 [    35  20000]
 [    26  43000]
 [    27  57000]
 [    19  76000]
 [    45 101000]
 [    39  59000]
 [    46  72000]
 [    28  82000]
 [    36  46000]
 [    33  58000]
 [    42 142000]
 [    29  90000]
 [    29  92000]
 [    25  88000]
 [    40  99000]
 [    38  79000]
 [    33  48000]
 [    28 102000]
 [    44  94000]
 [    33  99000]
 [    28  86000]
 [    33  75000]
 [    31  57000]
 [    43  54000]
 [    44  66000]
 [    20  20000]
 [    28  57000]
 [    45  38000]
 [    46  15000]
```

[31 20000]
[38 124000]
[39 77000]
[53 29000]
[33 23000]
[27 80000]
[60 84000]
[29 55000]
[49 75000]
[32 68000]
[48 53000]
[60 42000]
[47 83000]
[42 54000]
[39 123000]
[51 77000]
[42 122000]
[55 128000]
[53 32000]
[37 64000]
[38 91000]
[25 40000]
[24 71000]
[34 104000]
[36 46000]
[26 73000]
[57 40000]
[41 110000]
[18 64000]
[32 40000]
[22 142000]
[60 30000]
[56 36000]
[34 65000]
[35 77000]
[39 81000]
[30 80000]
[54 83000]
[40 32000]
[19 101000]
[33 78000]
[35 75000]
[35 76000]
[41 128000]
[34 67000]
[44 128000]
[56 84000]
[30 103000]
[35 109000]
[21 67000]
[53 91000]
[25 47000]
[50 90000]
[42 124000]

[44 16000]
[32 97000]
[40 79000]
[42 103000]
[53 44000]
[38 115000]
[43 149000]
[33 73000]
[24 15000]
[40 21000]
[33 85000]
[39 95000]
[30 63000]
[43 60000]
[18 25000]
[45 60000]
[51 67000]
[38 41000]
[28 57000]
[42 34000]
[18 101000]
[31 84000]
[41 114000]
[34 107000]
[41 136000]
[49 41000]
[41 41000]
[33 60000]
[27 81000]
[44 58000]
[38 150000]
[44 68000]
[42 41000]
[49 82000]
[34 101000]
[22 116000]
[22 15000]
[32 85000]
[39 67000]
[30 100000]
[28 115000]
[53 74000]
[34 38000]
[44 51000]
[29 15000]
[35 64000]
[45 92000]
[59 67000]
[27 27000]
[37 81000]
[47 95000]
[39 20000]
[28 76000]
[35 82000]

[34 33000]
[29 78000]
[40 131000]
[24 55000]
[40 15000]
[31 48000]
[42 15000]
[23 85000]
[45 78000]
[38 79000]
[32 56000]
[54 150000]
[34 60000]
[43 19000]
[39 58000]
[24 15000]
[57 64000]
[41 40000]
[34 15000]
[37 15000]
[32 15000]
[24 98000]
[42 84000]
[44 112000]
[32 55000]
[33 55000]
[47 15000]
[27 38000]
[35 34000]
[28 88000]
[37 93000]
[25 16000]
[27 66000]
[28 119000]
[26 31000]
[36 79000]
[46 15000]
[27 15000]
[56 87000]
[34 35000]
[49 38000]
[51 91000]
[41 15000]
[55 128000]
[52 70000]
[38 25000]
[56 100000]
[35 33000]
[24 87000]
[48 106000]
[29 33000]
[49 65000]
[40 63000]
[31 80000]

[40 31000]
[41 30000]
[38 71000]
[42 123000]
[30 39000]
[34 134000]
[37 74000]
[30 102000]
[47 71000]
[42 15000]
[22 68000]
[23 56000]
[40 35000]
[18 69000]
[37 77000]
[26 79000]
[18 15000]
[38 59000]
[28 86000]
[36 124000]
[35 26000]
[42 88000]
[50 118000]
[34 40000]
[36 133000]
[38 70000]
[55 80000]
[47 99000]
[43 110000]
[18 83000]
[42 86000]
[32 146000]
[54 74000]
[23 61000]
[38 60000]
[27 34000]
[23 85000]
[45 82000]
[41 59000]
[56 114000]
[46 74000]
[30 55000]
[41 92000]
[25 119000]
[27 35000]
[44 81000]
[44 15000]
[37 105000]
[32 86000]
[32 97000]
[48 93000]
[32 88000]
[24 63000]
[53 75000]

```
[    42  64000]
[    31  67000]
[    53  67000]
[    21  38000]
[    60  50000]
[    42  72000]
[    23  17000]
[    38  97000]
[    50 123000]
[    49  90000]
[    26  93000]
[    46 120000]
[    35  73000]
[    41  36000]
[    30  61000]
[    36  61000]
[    35  46000]
[    38  71000]
[    56  15000]
[    45  86000]
[    48  76000]
[    45  73000]
[    18  21000]
[    22  36000]
[    56  56000]
[    36  36000]
[    29  75000]
[    46  42000]
[    23  59000]
[    54 104000]
[    58 136000]
[    26  25000]
[    36  47000]
[    60  80000]
[    36  45000]
[    18  29000]
[    51  83000]
[    45  50000]
[    32  31000]
[    38  81000]
[    43  91000]
[    55  51000]
[    33  15000]
[    53  42000]
[    31  96000]
[    48  73000]
[    37  66000]
[    44 122000]
[    41  48000]
[    41  62000]
[    18  97000]
[    37 138000]
[    33  56000]
[    21  47000]
```

[38 15000]
[18 36000]
[34 29000]
[43 132000]
[22 110000]
[21 77000]
[21 65000]
[47 79000]
[42 92000]
[42 84000]
[60 81000]
[35 89000]
[45 135000]
[33 61000]
[19 99000]
[51 112000]
[46 83000]
[43 76000]
[50 15000]
[38 112000]
[28 47000]
[18 133000]
[44 22000]
[32 50000]
[44 69000]
[21 112000]
[60 103000]
[35 83000]
[28 98000]
[35 38000]
[40 67000]
[49 47000]
[53 65000]
[34 49000]
[36 56000]
[38 130000]
[30 40000]
[24 23000]
[39 108000]
[36 141000]
[55 85000]
[46 42000]
[41 76000]
[26 57000]
[25 94000]
[51 81000]
[18 68000]
[43 100000]
[40 45000]
[24 15000]
[33 106000]
[44 88000]
[25 138000]
[28 121000]

```
[    40  66000]
[    29  88000]
[    30  50000]
[    30  96000]
[    33  74000]
[    31  30000]
[    21  70000]
[    22  27000]
[    18  81000]
[    30  65000]
[    47  36000]
[    36  45000]
[    41  78000]
[    49 112000]
[    52  87000]
[    41  61000]
[    30  88000]
[    30  89000]
[    50  76000]
[    41 106000]
[    39  69000]
[    48  70000]
[    43  15000]
[    36 109000]
[    32 113000]
[    21  83000]
[    39 150000]
[    50  46000]
[    40  46000]
[    42  62000]
[    30  60000]
[    32 111000]
[    40  65000]
[    21  23000]
[    42  54000]
[    40  31000]
[    40 134000]
[    48  64000]
[    36  35000]
[    32  90000]
[    31 135000]
[    46  41000]
[    51  23000]
[    50  20000]
[    36  33000]
[    49  36000]]
[0 0 0 0 0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 0 1
 0 1 0 1 1 1 0 0 1 1 1 1 1 1 0 0 0 1 0 1 0 0 0 0 1 1 0 1 0 0 1 0 0 0 0 0 1
 0 1 1 1 1 0 1 0 1 1 1 0 1 1 0 1 1 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0 1 1 1 0 0
 0 0 0 1 1 0 1 0 0 0 0 1 0 1 1 0 0 0 1 0 1 0 0 1 1 0 0 0 0 1 0 0 0 1 0 1 0
 0 1 0 0 1 0 1 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 1 1 1 1 1 1
 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 0
 1 1 1 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 1 0 0 1 1 0 1 0 0 0 0 0 1 1 0 1 0
 1 1 0 0 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 0 1 0
```

```
 1 1 0 0 0 0 0 1 0 0 0 0 0 1 1 0 1 0 0 1 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 1 1
 0 0 1 0 0 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1
 0 0 1 1 0 1 0 1 0 0 1 1 0 1 0 1 0 0 1 0 1 0 0 0 1 1 1 1 0 1]
0.84375
0.7375
              precision    recall  f1-score   support

           0       0.83      0.90      0.86       250
           1       0.80      0.70      0.75       150

    accuracy                           0.82       400
   macro avg       0.82      0.80      0.81       400
weighted avg       0.82      0.82      0.82       400
```

EXPERIMENT NO: 20

Experiment to understand K-means clustering algorithm for a given data set.

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv('Mall_Customers.csv')
print(df.info())
print(df.head())
sns.pairplot(df)
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
print(Final.head())
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8).map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)").add_legend();
plt.show()
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
```

```
  model=KMeans(n_clusters=i)
  model.fit(features_el)
  wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

Output:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 200 entries, 0 to 199

Data columns (total 5 columns):

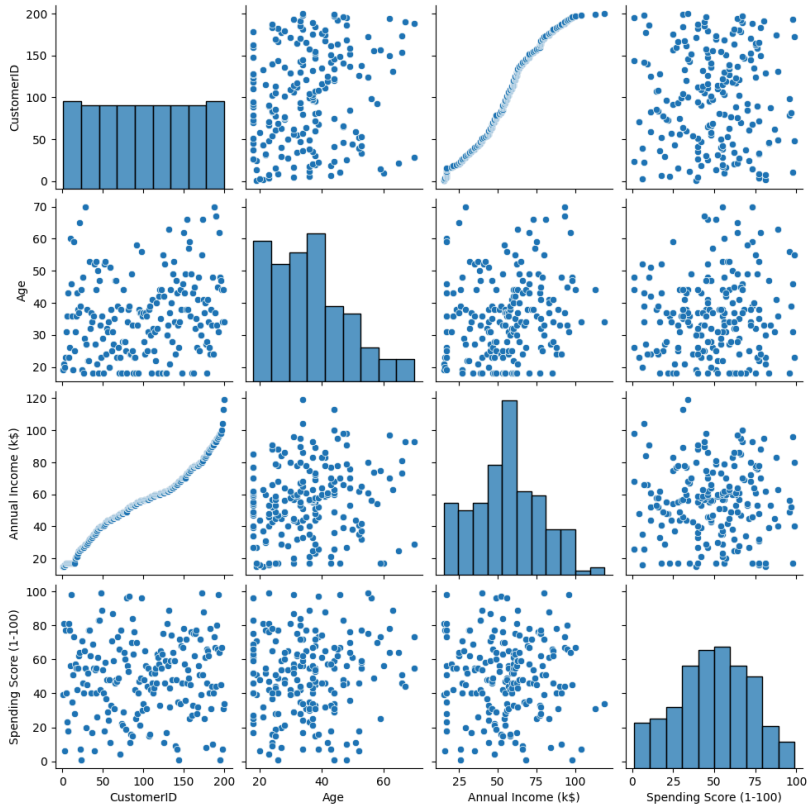
#	Column	Non-Null Count	Dtype
0	CustomerID	200 non-null	int64
1	Gender	200 non-null	object
2	Age	200 non-null	int64
3	Annual Income (k$)	200 non-null	int64
4	Spending Score (1-100)	200 non-null	int64

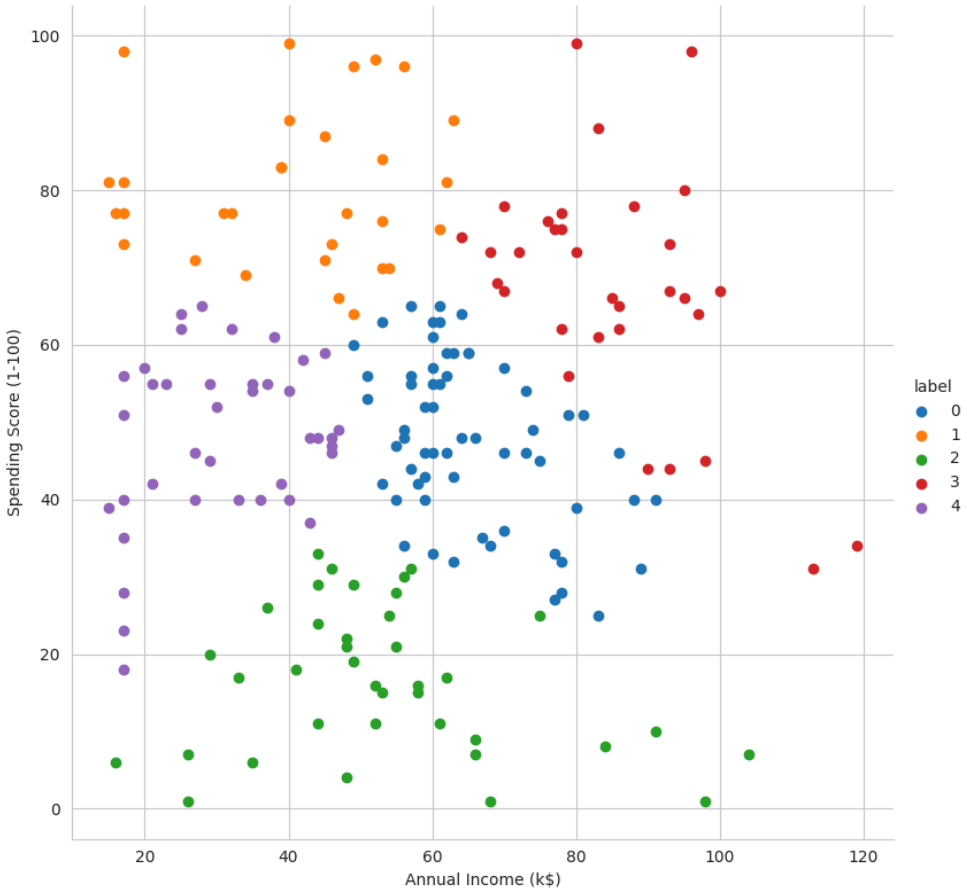
dtypes: int64(4), object(1)

0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
Final['label']=model.predict(features)
```

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	1
2	16	6	2
3	16	77	1
4	17	40	4





[<matplotlib.lines.Line2D at 0x7e90cd081810>]

